

## Memória cache

- 4.1** Visão geral do sistema de memória do computador
  - Características dos sistemas de memória
  - A hierarquia de memória
- 4.2** Princípios da memória cache
- 4.3** Elementos do projeto da memória cache
  - Endereços da cache
  - Tamanho da memória cache
  - Função de mapeamento
  - Algoritmos de substituição
  - Política de escrita
  - Tamanho da linha
  - Número de memórias caches
- 4.4** Organização da cache do Pentium 4
- 4.5** Organização da cache da ARM
- 4.6** Leitura recomendada
- Apêndice 4A** Características de desempenho de memórias de dois níveis
  - Localidade
  - Operação da memória de dois níveis
  - Desempenho

### PRINCIPAIS PONTOS

- A memória do computador é organizada em uma hierarquia. No nível mais alto (mais perto do processador), estão os registradores do processador. Em seguida, vêm um ou mais níveis de cache. Quando são usados múltiplos níveis, eles são indicados por L1, L2 e assim por diante. Em seguida, vem a memória principal, que normalmente é uma memória dinâmica de acesso aleatório e dinâmico (DRAM). Todos estes são considerados internos ao sistema de computação. A hierarquia continua com a memória externa, com o próximo nível geralmente sendo um disco rígido fixo, e um ou mais níveis abaixo disso consistindo em mídia removível, como discos ópticos e fita.
- À medida que descemos na hierarquia da memória, encontramos custo/bit menor, capacidade maior e tempo de acesso mais lento. Seria bom usar apenas a memória mais rápida, mas, como ela é a memória mais cara, trocamos tempo de acesso pelo custo, usando mais da memória mais lenta. O desafio de projeto é organizar os dados e os programas na memória de modo que as palavras de memória acessadas normalmente estejam na memória mais rápida.
- Em geral, é provável que a maioria dos acessos futuros à memória principal, feitos pelo processador, seja para locais acessados recentemente. Assim, a cache retém automaticamente uma cópia de algumas das palavras usadas recentemente, vindas das DRAM. Se a memória cache for projetada corretamente, então, na maior parte do tempo, o processador solicitará palavras da memória que já estão na cache.

Embora aparentemente simples em conceito, a memória do computador exibe talvez a gama mais variada de tipo, tecnologia, organização, desempenho e custo do que qualquer recurso de um sistema de computação. Como consequência, o sistema de computação típico é equipado com uma hierarquia de subsistemas de memória, algumas internas ao sistema (acessíveis diretamente pelo processador) e algumas externas (acessíveis pelo processador por meio de um módulo de E/S).

Este capítulo e o seguinte focalizam os elementos da memória interna, enquanto o Capítulo 6 é dedicado à memória externa. Para começar, a primeira seção examina as principais características das memórias de computador. O restante do capítulo examina um elemento essencial de todos os sistemas de computação modernos: a memória cache.

## 4.1 Visão geral do sistema de memória do computador

### Características dos sistemas de memória

O assunto complexo da memória de computador pode ser melhor compreendido se classificarmos os sistemas de memória de acordo com suas principais características. As mais importantes estão listadas na Tabela 4.1.

O termo **localização** na Tabela 4.1 indica se a memória é interna ou externa ao computador. A memória interna normalmente significa a memória principal, mas existem outras formas de memória interna. O processador requer sua própria memória local, na forma de registradores (por exemplo, veja a Figura 2.3). Além disso, conforme veremos, a parte da unidade de controle do processador também pode exigir sua própria memória interna. Vamos deixar a discussão desses dois últimos tipos de memória interna para capítulos posteriores. A cache é outra forma de memória interna. A memória externa consiste em dispositivos de armazenamento periféricos, como disco e fita, que são acessíveis ao processador por meio de controladores de E/S.

Uma característica óbvia da memória é a sua **capacidade**. Para a memória interna, isso normalmente é expresso em termos de bytes (1 byte = 8 bits) ou palavras. Os tamanhos comuns de palavra são 8, 16 e 32 bits. A capacidade da memória externa normalmente é expressa em termos de bytes.

Um conceito relacionado é a **unidade de transferência**. Para a memória interna, a unidade de transferência é igual ao número de linhas elétricas para dentro e para fora do módulo de memória. Isso pode ser igual ao tamanho da palavra, mas normalmente é maior, como 64, 128 ou 256 bytes. Para esclarecer esse ponto, considere três conceitos relacionados para a memória interna:

- **Palavra:** a unidade “natural” de organização da memória. O tamanho da palavra normalmente é igual ao número de bits usados para representar um inteiro e ao tamanho da instrução. Infelizmente, existem muitas exceções. Por exemplo, o CRAY C90 (um modelo de supercomputador CRAY mais antigo) tem um tamanho de palavra de 64 bits, mas usa a representação de inteiros com 46 bits. A arquitetura Intel x86 tem uma grande variedade de tamanhos de instrução, expressos como múltiplos de bytes e uma palavra com tamanho de 32 bits.
- **Unidades endereçáveis:** em alguns sistemas, a unidade endereçável é a palavra. Porém, muitos sistemas permitem o endereçamento no nível de byte. De qualquer forma, o relacionamento entre o tamanho em bits  $A$  de um endereço e o número  $N$  de unidades endereçáveis é  $2^A = N$ .

**Tabela 4.1** Principais características dos sistemas de memória do computador

| Localização                                                                   | Desempenho                     |
|-------------------------------------------------------------------------------|--------------------------------|
| Interna (por exemplo, registradores do processador, memória principal, cache) | Tempo de acesso                |
| Externa (por exemplo, discos ópticos, discos magnéticos, fitas)               | Tempo de ciclo                 |
| <b>Capacidade</b>                                                             | Taxa de transferência          |
| Número de palavras                                                            | <b>Tipo físico</b>             |
| Número de bytes                                                               | Semicondutor                   |
| <b>Unidade de transferência</b>                                               | Magnético                      |
| Palavra                                                                       | Óptico                         |
| Bloco                                                                         | Magneto-óptico                 |
| <b>Método de acesso</b>                                                       | <b>Características físicas</b> |
| Sequencial                                                                    | Volátil/não volátil            |
| Direto                                                                        | Apagável/não apagável          |
| Aleatório                                                                     | <b>Organização</b>             |
| Associativo                                                                   | Módulos de memória             |

- **Unidade de transferência:** para a memória principal, este é o número de bits lidos ou escritos na memória de uma só vez. A unidade de transferência não precisa ser igual a uma palavra ou uma unidade endereçável. Para a memória externa, os dados normalmente são transferidos em unidades muito maiores que uma palavra e estas são chamadas de blocos.

Outra distinção entre os tipos de memória é o **método de acesso** das unidades de dados, que inclui os seguintes:

- **Acesso sequencial:** a memória é organizada em unidades de dados chamadas registros. O acesso é feito em uma sequência linear específica. A informação de endereçamento armazenada é usada para separar registros e auxiliar no processo de recuperação. Um mecanismo compartilhado de leitura-escrita é usado, e este precisa ser movido do seu local atual para o local desejado, passando e rejeitando cada registro intermediário. Assim, o tempo para acessar um registro qualquer é altamente variável. As unidades de fita, discutidas no Capítulo 6, são de acesso sequencial.
- **Acesso direto:** assim como o acesso sequencial, o acesso direto envolve um mecanismo compartilhado de leitura-escrita compartilhado. Porém, os blocos ou registros individuais têm um endereço exclusivo, baseado no local físico. O acesso é realizado pelo acesso direto, para alcançar uma vizinhança geral, mais uma busca sequencial, contagem ou espera, até alcançar o local final. Novamente, o tempo de acesso é variável. As unidades de disco, discutidas no Capítulo 6, são de acesso direto.
- **Acesso aleatório:** cada local endereçável na memória tem um mecanismo de endereçamento exclusivo, fisicamente interligado. O tempo para acessar determinado local é independente da sequência de acessos anteriores e é constante. Assim, qualquer local pode ser selecionado aleatoriamente, e endereçado e acessado diretamente. A memória principal e alguns sistemas de cache são de acesso aleatório.
- **Associativo:** esse é o tipo de memória de acesso aleatório que permite fazer uma comparação de um certo número de bit desejados dentro de uma palavra para uma combinação especificada, e faz isso para todas as palavras simultaneamente. Assim, uma palavra é recuperada com base em uma parte de seu conteúdo, em vez do seu endereço. Assim como a memória de acesso aleatório comum, cada local tem seu próprio mecanismo de endereçamento, e o tempo de recuperação é constante, independentemente do local ou padrões de acesso anteriores. As memórias cache podem empregar o acesso associativo.

Do ponto de vista do usuário, as duas características mais importantes da memória são capacidade e **desempenho**. Três parâmetros de desempenho são usados:

- **Tempo de acesso (latência):** para a memória de acesso aleatório, esse é o tempo gasto para realizar uma operação de leitura ou escrita, ou seja, o tempo desde o instante em que um endereço é apresentado à memória até o instante em que os dados foram armazenados ou se tornaram disponíveis para uso. Para a memória de acesso não aleatório, o tempo de acesso é o tempo gasto para posicionar o mecanismo de leitura-escrita no local desejado.
- **Tempo de ciclo de memória:** esse conceito é aplicado principalmente à memória de acesso aleatório, e consiste no tempo de acesso mais qualquer tempo adicional antes que um segundo acesso possa iniciar. Esse tempo adicional pode ser exigido para a extinção de transientes nas linhas de sinal ou para a regeneração de dados, se eles forem lidos destrutivamente. Observe que o tempo de ciclo de memória se refere ao barramento do sistema, e não do processador.
- **Taxa de transferência:** essa é a taxa em que os dados podem ser transferidos para dentro ou fora de uma unidade de memória. Para a memória de acesso aleatório, ela é igual a  $1/(\text{tempo de ciclo})$ .

Para a memória de acesso não aleatório, existe a seguinte relação:

$$T_N = T_A + \frac{n}{R} \quad (4.1)$$

onde

$T_N$  = tempo médio para ler ou escrever  $N$  bits

$T_A$  = tempo de acesso médio

$n$  = número de bits

$R$  = taxa de transferência em bits por segundo (bps)

Diversas **tecnologias** de memória foram empregados. As mais comuns hoje são memória semicondutora, memória de superfície magnética, usada para disco e fita, e óptica e magneto-óptica.

Várias **características físicas** de armazenamento de dados são importantes. Em uma memória volátil, a informação se deteriora naturalmente ou se perde quando a energia elétrica é desligada. Em uma memória não volátil, a informação uma vez gravada permanece sem deterioração até que seja deliberadamente mudada; nenhuma energia elétrica é necessária para reter a informação. As memórias com superfície magnética são não voláteis. A memória semicondutora pode ser volátil ou não. A memória não apagável não pode ser alterada, exceto destruindo-se a unidade de armazenamento. A memória semicondutora desse tipo é conhecida como **memória somente de leitura** (ROM, do inglês *read-only memory*). Inevitavelmente, uma memória prática não apagável também precisa ser não volátil.

Para a memória somente de leitura, a **organização** é um aspecto fundamental de projeto. Com **organização**, queremos dizer o arranjo físico de bits para formar palavras. O arranjo óbvio nem sempre é usado, conforme explicamos no Capítulo 5.



## A hierarquia de memória

As restrições de projeto sobre a memória de um computador podem ser resumidas por três questões: Quanto? Com que velocidade? Com que custo?

A questão da quantidade é, de certa forma, livre. Se houver capacidade, as aplicações provavelmente serão desenvolvidas para utilizá-la. A questão da velocidade, de certa forma, é mais fácil de responder. Para conseguir maior desempenho, a memória precisa ser capaz de acompanhar a velocidade do processador. Ou seja, enquanto o processador está executando instruções, não gostaríamos que ele tivesse que parar, aguardando por instruções ou operandos. A questão final também precisa ser considerada. Para um sistema prático, o custo da memória deve ser razoável em relação a outros componentes.

Como se pode esperar, existe uma relação entre as três características principais da memória, a saber: capacidade, tempo de acesso e custo. Diversas tecnologias são usadas para implementar sistemas de memória e, por meio desse espectro de tecnologias, existem as seguintes relações:

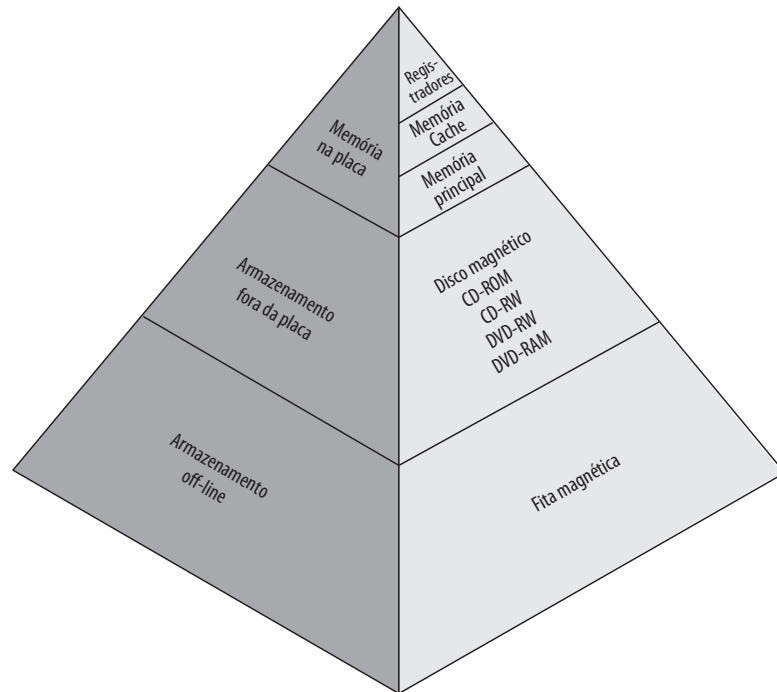
- Tempo de acesso mais rápido, maior custo por bit.
- Maior capacidade, menor custo por bit.
- Maior capacidade, tempo de acesso mais lento.

O dilema que o projetista enfrenta é claro. O projetista gostaria de usar tecnologias de memória que oferecessem grande capacidade de memória, tanto porque a capacidade é necessária quanto porque o custo por bit é baixo. Porém, para atender os requisitos de desempenho, ele precisa usar memórias caras, relativamente com menor capacidade e com menores tempos de acesso.

Para sair desse dilema, é preciso não contar com um único componente ou tecnologia de memória, mas empregar uma **hierarquia de memória**. Uma hierarquia típica é ilustrada na Figura 4.1. Enquanto se desce na hierarquia, ocorre o seguinte:

- a. Diminuição do custo por bit.
- b. Aumento da capacidade.
- c. Aumento do tempo de acesso.
- d. Frequência de acesso à memória pelo computador.

Assim, memórias menores, mais caras e mais rápidas são complementadas por memórias maiores, mais baratas e mais lentas. A chave para o sucesso dessa organização é o item (d): diminuição na frequência de acesso. Veremos esse conceito com mais detalhes quando discutirmos sobre a memória cache, mais adiante neste capítulo, e a memória virtual, no Capítulo 8. Neste ponto, oferecemos uma rápida explicação.

**Figura 4.1** A hierarquia de memória

**Exemplo 4.1** Suponha que o processador tenha acesso a dois níveis de memória. O nível 1 contém 1.000 palavras e tem um tempo de acesso de  $0,01 \mu\text{s}$ ; o nível 2 contém 100.000 palavras e tem um tempo de acesso de  $0,1 \mu\text{s}$ . Suponha que, se uma palavra a ser acessada estiver no nível 1, então o processador a acessa diretamente. Se estiver no nível 2, então a palavra primeiro é transferida para o nível 1 e depois é acessada pelo processador. Para simplificar, ignoramos o tempo necessário para o processador determinar se a palavra está no nível 1 ou no nível 2. A Figura 4.2 mostra o formato geral da curva que abrange essa situação. A figura mostra o tempo médio de acesso para uma memória de dois níveis como uma função da razão de acerto  $H$ , onde  $H$  é definido como a fração de todos os acessos à memória que são encontrados na memória mais rápida (por exemplo, a cache),  $T_1$  é o tempo de acesso ao nível 1, e  $T_2$  é o tempo de acesso ao nível 2.<sup>1</sup> Como podemos ver, para altas percentagens de acesso ao nível 1, o tempo médio de acesso total é muito mais próximo daquele do nível 1 do que do nível 2.

Em nosso exemplo, suponha que 95% dos acessos à memória sejam encontrados na cache. Então, o tempo médio para acessar uma palavra pode ser expresso como:

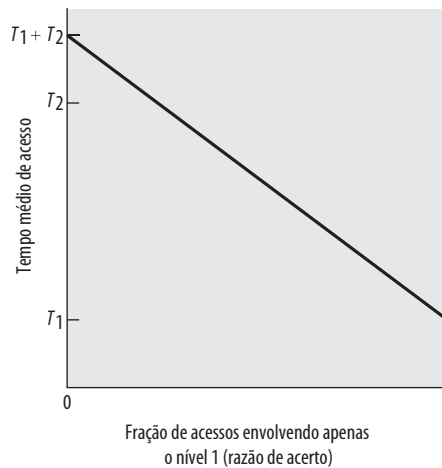
$$(0,95)(0,01 \mu\text{s}) + (0,05)(0,01 \mu\text{s} + 0,1 \mu\text{s}) = 0,0095 + 0,0055 = 0,015 \mu\text{s}$$

O tempo de acesso médio é muito mais próximo de  $0,01 \mu\text{s}$  do que de  $0,1 \mu\text{s}$ , conforme desejado.

O uso de dois níveis de memória para reduzir o tempo médio de acesso funciona em princípio, mas somente se as condições de (a) a (d) se aplicarem. Empregando diferentes tecnologias, existe um espectro de sistemas de memória que satisfaz as condições de (a) a (c). Felizmente, a condição (d) também geralmente é válida.

A base para a validade da condição (d) é um princípio conhecido como **localidade de referência** (DENNING, 1968<sup>a</sup>). Durante o curso de execução de um programa, as referências de memória pelo processador, para instruções e para dados, tendem a se agrupar. Os programas normalmente contêm uma série de loops iterativos

<sup>1</sup> Se a palavra acessada for encontrada na memória mais rápida, tem-se um **acerto**. Uma **falha** ocorre se a palavra acessada não for encontrada na memória mais rápida.

**Figura 4.2** Desempenho dos acessos envolvendo apenas o nível 1 (razão de acerto)

e sub-rotinas. Quando um loop ou sub-rotina inicia sua execução, existem referências repetidas a um pequeno conjunto de instruções. De modo semelhante, operações sobre tabelas e arrays envolvem o acesso a um conjunto de palavras de dados agrupadas. Durante um longo período de tempo os conjuntos mudam, mas para um pequeno período de tempo o processador trabalha com conjuntos fixos de referências à memória.

De forma correspondente, é possível organizar dados pela hierarquia de modo que a percentagem de acessos a cada nível sucessivamente inferior é muito menor que para o nível acima. Considere o exemplo de dois níveis, já apresentado. Suponha que a memória de nível 2 contenha todas as instruções e dados do programa. Os conjuntos atuais podem ser temporariamente colocados no nível 1. De vez em quando, um dos conjuntos no nível 1 terá que ser passado para o nível 2, para dar espaço para um novo conjunto chegando ao nível 1. Porém, na média, a maioria das referências será para instruções e dados contidos no nível 1.

Esse princípio pode ser aplicado por mais de dois níveis de memória, conforme sugerido pela hierarquia mostrada na Figura 4.1. O tipo de memória mais rápido, menor e mais caro consiste nos registradores internos ao processador. Normalmente, um processador terá algumas dezenas desses registradores, embora algumas máquinas contenham centenas de registradores. Pulando dois níveis abaixo, a memória principal é o principal sistema de memória interna do computador. Cada local na memória principal tem um endereço exclusivo. A memória principal normalmente é estendida com uma memória cache menor, de maior velocidade. A cache normalmente não é visível ao programador ou, na verdade, ao processador. Esse é um dispositivo para organizar a movimentação de dados entre a memória principal e os registradores do processador, para melhorar o desempenho.

As três formas de memória que descrevemos normalmente são voláteis e empregam a tecnologia semicondutora. O uso de três níveis explora o fato de que existem diversos tipos de memória semicondutora vem em diversos tipos, que diferem em velocidade e custo. Os dados são armazenados de forma mais permanente em dispositivos externos, de armazenamento em massa, sendo os mais comuns o disco rígido e a mídia removível, como disco magnético removível, fita e armazenamento óptico. A memória externa, não volátil, também é chamada de **memória secundária** ou **memória auxiliar**. Estas são usadas para armazenar arquivos de programa e dados e, normalmente, são visíveis ao programador apenas em termos de arquivos e registros, ao contrário de bytes ou palavras individuais. O disco também é usado para oferecer uma extensão à memória principal, conhecida como memória virtual, que será discutida no Capítulo 8.

Outras formas de memória podem ser incluídas na hierarquia. Por exemplo, grandes mainframes IBM incluem uma forma de memória interna conhecida como armazenamento expandido. Este usa uma tecnologia de semicondutora que é mais lenta e menos dispendiosa do que a da memória principal. Estritamente falando, essa memória não se encaixa na hierarquia, mas é um apêndice: os dados podem ser movidos entre a memória principal e o armazenamento expandido, mas não entre o armazenamento expandido e a memória externa. Outras formas de memória secundária incluem os discos ópticos e magneto-ópticos. Finalmente, outros níveis podem ser efeti-

vamente introduzidos à hierarquia por meio do uso de software. Uma parte da memória principal pode ser usada como um buffer para manter temporariamente os dados que devem ser levados ao disco. Essa técnica, às vezes chamada de cache de disco,<sup>2</sup> melhora o desempenho de duas maneiras:

- As gravações em disco são agrupadas. Em vez de muitas transferências de dados pequenas, temos algumas transferências de dados grandes. Isso melhora o desempenho do disco e minimiza o envolvimento do processador.
- Alguns dados destinados para escrita podem ser referenciados por um programa antes da próxima cópia no disco. Nesse caso, os dados são recuperados rapidamente da cache de disco, ao invés de lentamente do disco.

O Apêndice 4A examina as implicações de desempenho das estruturas de memória multinível.



## 4.2 Princípios da memória cache

O uso da memória cache visa obter velocidade de memória próxima das memórias mais rápidas que existem e, ao mesmo tempo, disponibilizar uma memória de grande capacidade ao preço de memórias semicondutoras mais baratas. O conceito é ilustrado na Figura 4.3a. Existe uma memória principal relativamente grande e lenta junto com a memória cache, menor e mais rápida. A cache contém uma cópia de partes da memória principal. Quando o processador tenta ler uma palavra da memória, é feita uma verificação para determinar se a palavra está na cache. Se estiver, ela é entregue ao processador. Se não, um bloco da memória principal, consistindo em algum número fixo de palavras, é lido para a cache e depois a palavra é fornecida ao processador. Devido ao fenômeno de localidade de referência, quando um bloco de dados é levado para a cache para satisfazer uma única referência de memória, é provável que haja referências futuras a esse mesmo local da memória ou a outras palavras no mesmo bloco.

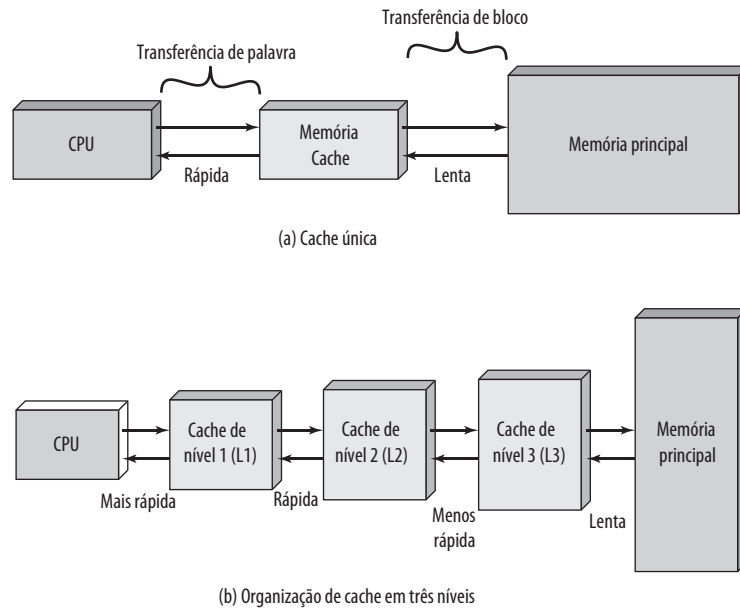
A Figura 4.3b representa o uso de múltiplos níveis de cache. A cache L2 é mais lenta e normalmente maior que a cache L1, e a cache L3 é mais lenta e normalmente maior que a cache L2.

A Figura 4.4 representa a estrutura de um sistema de cache/memória principal. A memória principal consiste em até  $2^n$  palavras endereçáveis, com cada palavra tendo um endereço distinto de  $n$  bits. Para fins de mapeamento, essa memória é considerada como sendo uma série de blocos de tamanho fixo com  $K$  palavras cada. Ou seja, existem  $M = 2^n/K$  blocos na memória principal. A cache consiste em  $m$  blocos, chamados **linhas**.<sup>3</sup> Cada linha contém  $K$  palavras, mais um tag de alguns bits. Cada linha também inclui bits de controle (não mostrados), como um bit para indicar se a linha foi modificada desde que foi carregada na cache. A largura de uma linha, sem incluir tag e bits de controle, é o tamanho da linha. O tamanho da linha pode ter apenas 32 bits, com cada “palavra” sendo um único byte; nesse caso, o tamanho da linha é de 4 bytes. O número de linhas é consideravelmente menor que o número de blocos da memória principal ( $m \ll M$ ). A qualquer momento, algum subconjunto dos blocos de memória reside nas linhas na cache. Se uma palavra em um bloco de memória for lida, esse bloco é transferido para uma das linhas da cache. Como existem mais blocos do que linhas, uma linha individual não pode ser dedicada exclusiva e permanentemente a determinado bloco. Assim, cada linha inclui uma tag que identifica qual bloco em particular está atualmente sendo armazenado. O tag normalmente é uma parte do endereço da memória principal, conforme descrito posteriormente nesta seção.

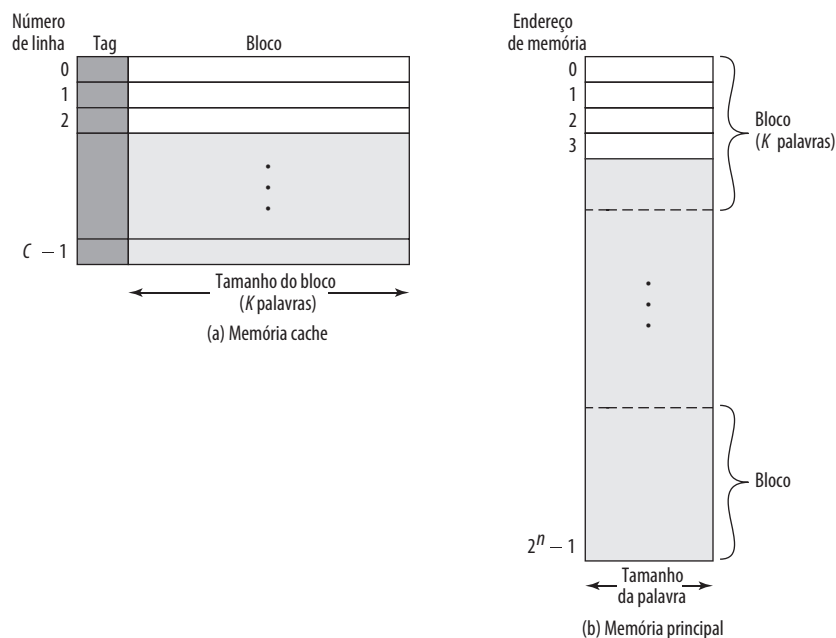
A Figura 4.5 ilustra a operação de leitura. O processador gera o endereço de leitura (RA, do inglês *read address*) de uma palavra a ser lida. Se a palavra estiver na cache, ela é entregue ao processador. Caso contrário, o bloco contendo essa palavra é carregado na cache e a palavra é entregue ao processador. A Figura 4.5 mostra essas duas operações ocorrendo em paralelo e reflete a organização mostrada na Figura 4.6, que é típica das organizações de cache modernas. Nessa organização, a cache se conecta ao processador por meio de linhas de dados, controle e endereço. As linhas de dados e endereços também se conectam a buffers de dados e endereços, que se conectam a um barramento do sistema, do qual a memória principal é acessada. Quando ocorre um acerto de cache (*cache hit*),

2 A cache de disco geralmente é uma técnica puramente do software, e não é examinada neste livro. Veja uma discussão em Stallings (2009<sup>b</sup>).

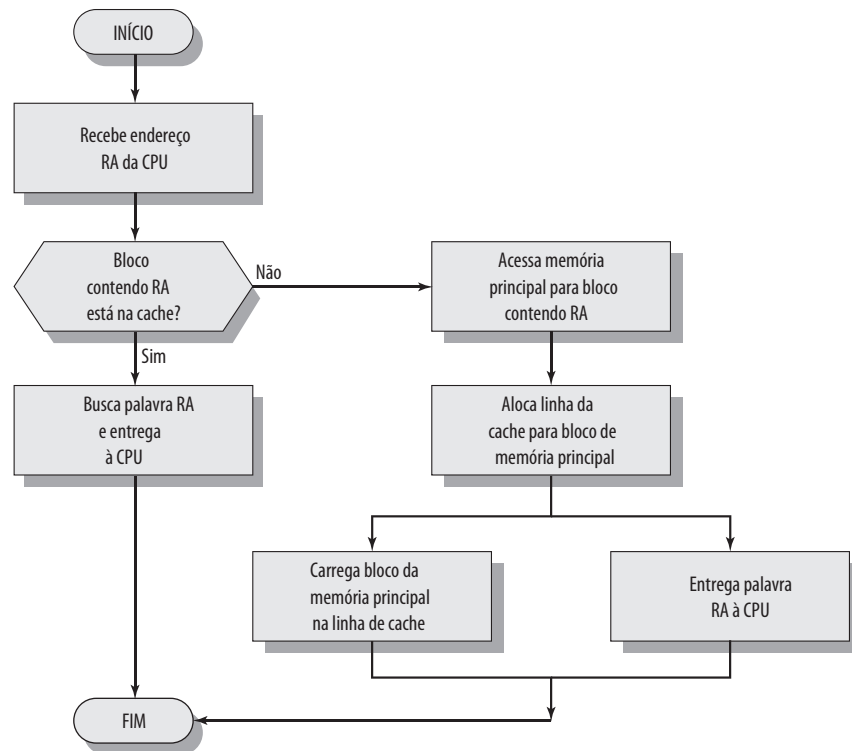
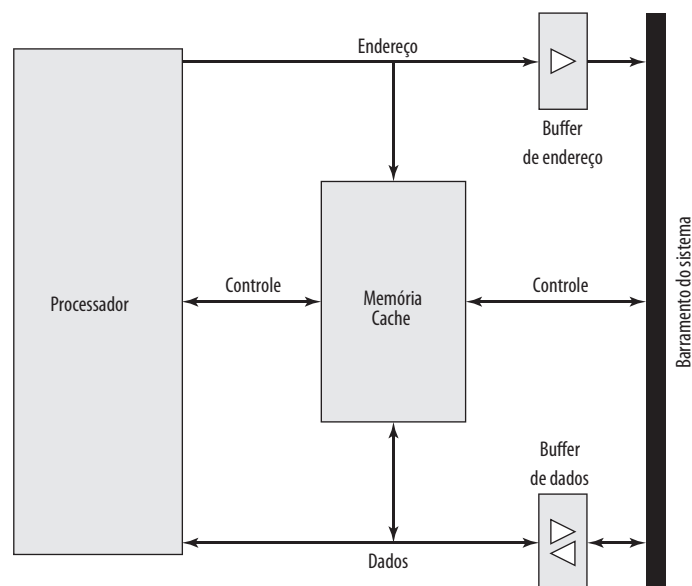
3 Referindo-se à unidade básica da cache, o termo linha, em vez de bloco, é usado por dois motivos: (1) para evitar confusão com um bloco da memória principal, que contém o mesmo número de palavras de dados que uma linha de cache; e (2) porque uma linha de cache inclui não apenas  $K$  palavras de dados, como um bloco da memória principal, mas também inclui tag e bits de controle.

**Figura 4.3** Cache e memória principal

os buffers de dados e endereço são desativados e a comunicação é apenas entre o processador e a memória cache, sem tráfego no barramento do sistema. Quando ocorre uma falha de cache (*cache miss*), o endereço desejado é carregado no barramento do sistema e os dados são transferidos através do buffer de dados para a cache e para o processador. Em outras organizações, a cache é fisicamente interposta entre o processador e a memória principal para todas as linhas de dados, endereço e controle são ligadas à cache. Nesse último caso, para uma falha de cache, a palavra desejada primeiro é lida para a cache e depois transferida da cache para o processador.

**Figura 4.4** Estrutura de cache/memória principal



**Figura 4.5** Operação de leitura de cache**Figura 4.6** Organização típica da memória cache

Uma discussão sobre os parâmetros de desempenho relacionados ao uso da cache pode ser vista no Apêndice 4A.



### 4.3 Elementos do projeto da memória cache

Esta seção oferece uma visão geral dos parâmetros de projeto de memória cache e informa alguns resultados típicos. Ocasionalmente, nos referimos ao uso de caches na computação de alto desempenho (HPC, do inglês *high performance computing*). A HPC lida com supercomputadores e software para supercomputador, especialmente para aplicações científicas, que envolvem grandes quantidades de dados, cálculos de vetores e matrizes e uso de algoritmos paralelos. O projeto de memória cache para HPC é muito diferente daquele para outras plataformas de hardware e aplicações. Na verdade, muitos pesquisadores descobriram que aplicações HPC não funcionam bem em arquiteturas de computador que empregam memórias caches (BAIL, 1993<sup>c</sup>). Outros pesquisadores, desde então, têm demonstrado que uma hierarquia de memória cache pode ser útil para melhorar o desempenho se o software de aplicação for ajustado para explorar a cache (WANG e TAFTI, 1999<sup>d</sup>); (PRESSEL, 2001<sup>e</sup>).<sup>4</sup>

Embora haja um grande número de implementações de memória cache, existem alguns elementos básicos de projeto que servem para classificar e diferenciar as arquiteturas de memórias cache. A Tabela 4.2 lista os principais elementos.



#### Endereços de cache

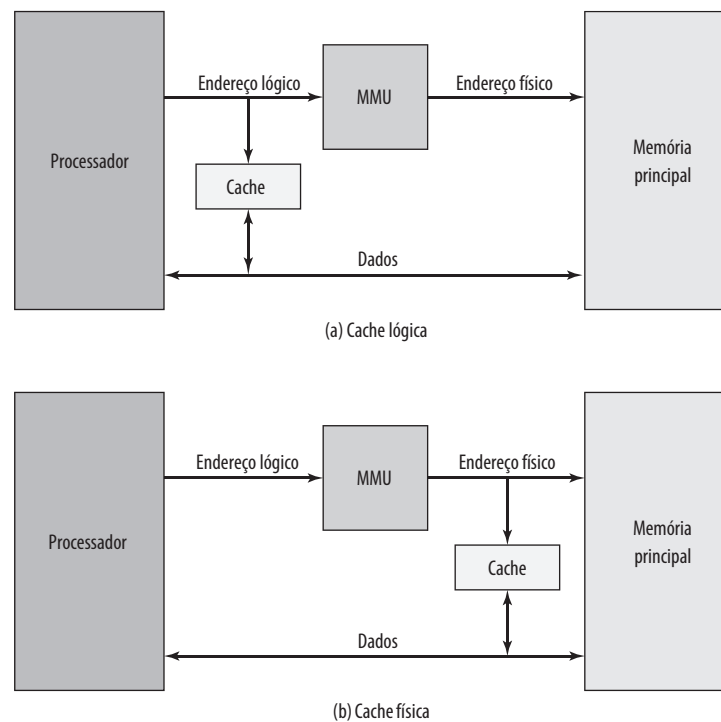
Quase todos os processadores não embutidos, e muitos processadores embutidos, admitem memória virtual, um conceito discutido no Capítulo 8. Basicamente, a memória virtual é uma facilidade que permite que os programas enderecem a memória a partir de um ponto de vista lógico, sem considerar a quantidade de memória principal disponível fisicamente. Quando a memória virtual é usada, os campos de endereço das instruções de máquina contêm endereços virtuais. Para leituras e escritas da memória principal, uma unidade de gerenciamento da memória (MMU, do inglês *memory management unit*) física traduz cada endereço virtual para um endereço físico na memória principal.

Quando são usados endereços virtuais, o projetista do sistema pode escolher colocar a cache entre o processador e a MMU ou entre a MMU e a memória principal (Figura 4.7). Uma **cache lógica**, também conhecida como **cache virtual**, armazena dados usando **endereços virtuais**. O processador acessa a cache diretamente, sem passar pela MMU. Uma cache física armazena dados usando **endereços físicos** da memória principal.

**Tabela 4.2** Elementos do projeto de cache

| Endereços de cache                                                              | Política de escrita   |
|---------------------------------------------------------------------------------|-----------------------|
| Lógicos                                                                         | <i>Write-through</i>  |
| Físicos                                                                         | <i>Write-back</i>     |
| Tamanho de cache                                                                | <i>Write once</i>     |
| Função de mapeamento                                                            | Tamanho de linha      |
| Direta                                                                          | Número de caches      |
| Associativa                                                                     | Um ou dois níveis     |
| Associativa em conjunto ( <i>set associative</i> )                              | Unificada ou separada |
| Algoritmo de substituição                                                       |                       |
| Usado menos recentemente (LRU, do inglês <i>least recently used</i> )           |                       |
| Primeiro a entrar, primeiro a sair (FIFO, do inglês <i>first-in-first-out</i> ) |                       |
| Usado menos frequentemente (LFU, do inglês <i>least frequently used</i> )       |                       |
| Aleatório                                                                       |                       |

<sup>4</sup> Para ver uma discussão geral sobre HPC, consulte Dowd e Severance (1998<sup>f</sup>).

**Figura 4.7** Caches lógicas e físicas

Uma vantagem óbvia da cache lógica é que a velocidade de acesso a ela é maior do que para uma cache física, pois a cache pode responder antes que a MMU realize uma tradução de endereço. A desvantagem é que a maioria dos sistemas de memória virtual fornece, a cada aplicação, o mesmo espaço de endereços de memória virtual. Ou seja, cada aplicação vê uma memória virtual que começa no endereço 0. Assim, o mesmo endereço virtual em duas aplicações diferentes refere-se a dois endereços físicos diferentes. A memória cache, portanto, precisa ser completamente esvaziada a cada troca de contexto de aplicação, ou então bits extras precisam ser adicionados a cada linha da cache para identificar a que espaço de endereço virtual esse endereço se refere.

O assunto de cache lógica *versus* física é complexo, e está fora do escopo deste livro. Para obter uma discussão mais profunda, consulte Cekanov e Dubois (1997<sup>9</sup>) e Jacob, Ng e Wang (2008<sup>h</sup>).



### Tamanho da memória cache

O primeiro item na Tabela 4.2, o tamanho da memória cache, já foi discutido. Gostaríamos que o tamanho da cache fosse pequeno o suficiente para que o custo médio geral por bit fosse próximo do custo médio da memória principal isolada e grande o suficiente para que o tempo de acesso médio geral fosse próximo do tempo de acesso médio da cache isolada. Existem várias outras motivações para minimizar o tamanho da cache. O resultado é que caches grandes tendem a ser ligeiramente mais lentas que as pequenas — mesmo quando construídas com a mesma tecnologia de circuito integrado e colocadas no mesmo lugar no chip e na placa de circuito. A área disponível do chip e da placa também limita o tamanho da cache. Como o desempenho da cache é muito sensível à natureza da carga de trabalho, é impossível chegar a um único tamanho ideal de cache. A Tabela 4.3 lista os tamanhos de cache de alguns processadores atuais e antigos.

**Tabela 4.3** Tamanhos de memória cache de alguns processadores

| Processador           | Tipo                                  | Ano de introdução | Cache L1 <sup>a</sup> | Cache L2      | Cache L3   |
|-----------------------|---------------------------------------|-------------------|-----------------------|---------------|------------|
| IBM 360/85            | Mainframe                             | 1968              | 16 a 32 KB            | —             | —          |
| PDP-11/70             | Minicomputador                        | 1975              | 1 KB                  | —             | —          |
| VAX 11/780            | Minicomputador                        | 1978              | 16 KB                 | —             | —          |
| IBM 3033              | Mainframe                             | 1978              | 64 KB                 | —             | —          |
| IBM 3090              | Mainframe                             | 1985              | 128 a 256 KB          | —             | —          |
| Intel 80486           | PC                                    | 1989              | 8 KB                  | —             | —          |
| Pentium               | PC                                    | 1993              | 8 KB/8 KB             | 256 a 512 KB  | —          |
| PowerPC 601           | PC                                    | 1993              | 32 KB                 | —             | —          |
| PowerPC 620           | PC                                    | 1996              | 32 KB/32 KB           | —             | —          |
| PowerPC G4            | PC/servidor                           | 1999              | 32 KB/32 KB           | 256 KB a 1 MB | 2 MB       |
| IBM S/390 G4          | Mainframe                             | 1997              | 32 KB                 | 256 KB        | 2 MB       |
| IBM S/390 G6          | Mainframe                             | 1999              | 256 KB                | 8 MB          | —          |
| Pentium 4             | PC/servidor                           | 2000              | 8 KB/8 KB             | 256 KB        | —          |
| IBM SP                | Servidor avançado/<br>Supercomputador | 2000              | 64 KB/32 KB           | 8 MB          | —          |
| CRAY MTA <sup>b</sup> | Supercomputador                       | 2000              | 8 KB                  | 2 MB          | —          |
| Itanium               | PC/servidor                           | 2001              | 16 KB/16 KB           | 96 KB         | 4 MB       |
| SGI Origin 2001       | Servidor avançado                     | 2001              | 32 KB/32 KB           | 4 MB          | —          |
| Itanium 2             | PC/servidor                           | 2002              | 32 KB                 | 256 KB        | 6 MB       |
| IBM POWER5            | Servidor avançado                     | 2003              | 64 KB                 | 1,9 MB        | 36 MB      |
| CRAY XD-1             | Supercomputador                       | 2004              | 64 KB/64 KB           | 1 MB          | —          |
| IBM POWER6            | PC/servidor                           | 2007              | 64 KB/64 KB           | 4 MB          | 32 MB      |
| IBM z10               | Mainframe                             | 2008              | 64 KB/128 KB          | 3 MB          | 24 a 48 MB |

**a** Dois valores separados por uma barra referem-se a caches de instrução e dados.

**b** As duas caches são apenas de instrução; não há caches de dados.



## Função de mapeamento

Como existem menos linhas de cache do que blocos da memória principal, é necessário haver um algoritmo para mapear os blocos da memória principal às linhas de cache. Além do mais, é preciso haver um meio para determinar qual bloco da memória principal atualmente ocupa uma linha da cache. A escolha da função de mapeamento dita como a cache é organizada. Três técnicas podem ser utilizadas: direta, associativa e associativa em conjunto (*set associative*). Vamos examinar cada uma por sua vez. Em cada caso, examinamos a estrutura geral e depois um exemplo específico.

**Exemplo 4.2** Para todos os três casos, o exemplo inclui os seguintes elementos:

- A cache pode manter 64 KBytes.
- Os dados são transferidos entre a memória principal e a cache em blocos de 4 bytes cada. Isso significa que a cache é organizada como  $16\text{ K} = 2^{14}$  linhas de 4 bytes cada.
- A memória principal consiste em 16 MBytes, com cada byte endereçável diretamente por um endereço de 24 bits ( $2^{24} = 16\text{ M}$ ). Assim, para fins de mapeamento, podemos considerar que a memória principal consiste em 4 M blocos de 4 bytes cada.

**MAPEAMENTO DIRETO** A técnica mais simples, conhecida como mapeamento direto, mapeia cada bloco da memória principal a apenas uma linha de cache possível. O mapeamento é expresso como:

$$i = j \text{ módulo } m$$

onde

$i$  = número da linha da cache

$j$  = número do bloco da memória principal

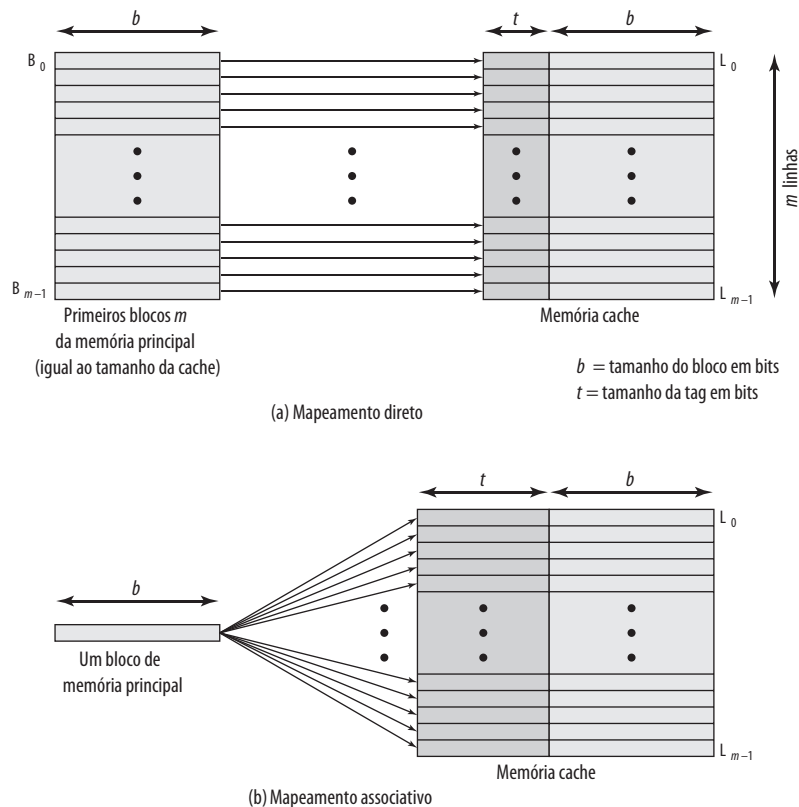
$m$  = número de linhas da cache

A Figura 4.8a mostra o mapeamento para os primeiros  $m$  blocos de memória principal. Cada bloco da memória principal mapeia uma linha exclusiva da cache. Os próximos  $m$  blocos da memória principal mapeiam a cache da mesma forma; ou seja, o bloco  $B_m$  da memória principal mapeia a linha  $L_0$  da cache, o bloco  $B_{m+1}$  mapeia a linha  $L_1$ , e assim por diante.

A função de mapeamento é facilmente implementada por meio do endereço da memória principal. A Figura 4.9 ilustra o mecanismo geral. Para fins de acesso à cache, cada endereço da memória principal pode ser visto como consistindo em três campos. Os  $w$  bits menos significativos identificam uma palavra ou um byte dentro de um bloco da memória principal; na maioria das máquinas modernas, o endereço está no nível de byte. Os  $s$  bits restantes especificam um dos  $2^s$  blocos da memória principal. A lógica de cache interpreta esses  $s$  bits como uma tag de  $s - r$  bits (parte mais significativa) e um campo de linha de  $r$  bits. O segundo campo identifica uma das  $m = 2^r$  linhas da cache. Resumindo,

- Tamanho do endereço =  $(s + w)$  bits.
- Número de unidades endereçáveis =  $2^{s+w}$  palavras ou bytes.
- Tamanho do bloco = tamanho da linha =  $2^w$  palavras ou bytes.
- Número de blocos na memória principal =  $\frac{2^{s+w}}{2^w} = 2^s$ .
- Número de linhas na cache =  $m = 2^r$ .
- Tamanho da cache =  $2^{r+w}$  palavras ou bytes.
- Tamanho da tag =  $(s - r)$  bits.

**Figura 4.8** Mapeamento da memória principal para a cache: direto e associativo



O efeito desse mapeamento é que os blocos da memória principal são alocados nas linhas da cache como mostrado na Tabela 4.4.

Assim, o uso de uma parte do endereço como o número da linha oferece um mapeamento exclusivo de cada bloco da memória principal à cache. Quando um bloco é armazenado na sua respectiva linha, é necessário marcar os dados para distingui-los de outros blocos que podem ser alocados nessa linha. Os  $s - r$  bits mais significativos têm essa finalidade.

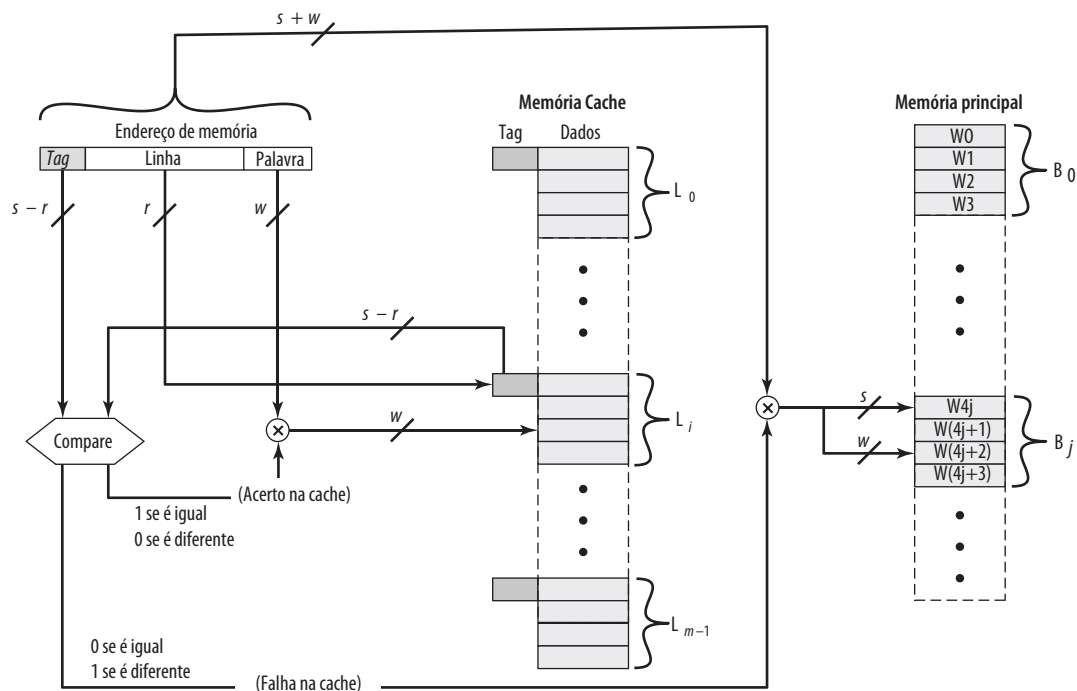
**Exemplo 4.2a** A Figura 4.10 mostra nosso sistema de exemplo usando o mapeamento direto.<sup>5</sup> No exemplo,  $m = 16\text{ K} = 2^{14}$  e  $i = j$  módulo  $2^{14}$ . O mapeamento torna-se

| Linha de cache | Endereço de memória inicial do bloco |
|----------------|--------------------------------------|
| 0              | 000000, 010000, ..., FF0000          |
| 1              | 000004, 010004, ..., FF0004          |
| $\vdots$       | $\vdots$                             |
| $2^{14} - 1$   | 00FFFC, 01FFFC, ..., FFFFFC          |

Observe que não existem dois blocos mapeados para o mesmo número de linha que tenham o mesmo número de tag. Assim, os blocos com endereços iniciais 000000, 010000, ..., FF0000 possuem números de tag 00, 01, ..., FF, respectivamente.

Retornando à Figura 4.5, uma operação de leitura funciona da seguinte forma. O sistema da memória cache recebe um endereço de 24 bits. O número de linha com 14 bits é usado como um índice para a cache acessar uma linha em particular. Se o número de tag com 8 bits for igual ao número de tag atualmente armazenado nessa linha, então o número da palavra com 2 bits é usado para selecionar um dos 4 bytes nessa linha. Caso contrário, o campo de tag-mais-linha com 22 bits é usado para buscar um bloco da memória principal. O endereço real que é usado para a busca é o campo de tag-mais-linha com 22 bits concatenado com dois bits 0, de modo que 4 bytes sejam apanhados a partir do início do bloco.

**Figura 4.9** Organização da memória com mapeamento direto

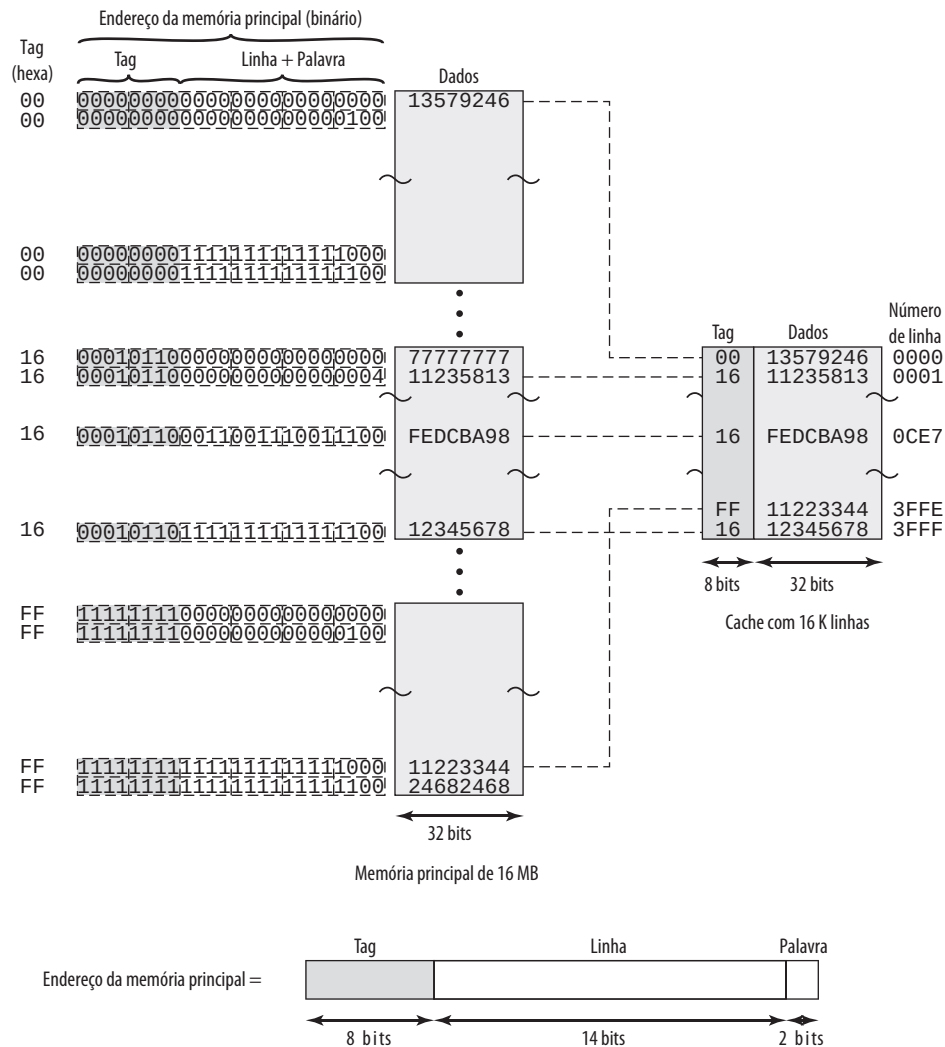


<sup>5</sup> Nesta e nas figuras seguintes, os valores de memória são representados em notação hexadecimal. Veja, no Capítulo 19, um manual básico sobre sistemas numéricos (decimal, binário, hexadecimal).

**Tabela 4.4** Mapeamento dos blocos da memória principal nas linhas da cache

| Linha de cache | Blocos de memória principal mapeados    |
|----------------|-----------------------------------------|
| 0              | $0, m, 2m, \dots, 2^s - m$              |
| 1              | $1, m + 1, 2m + 1, \dots, 2^s - m + 1$  |
| $\vdots$       | $\vdots$                                |
| $m - 1$        | $m - 1, 2m - 1, 3m - 1, \dots, 2^s - 1$ |

**Figura 4.10** Exemplo de mapeamento direto



Nota: Valores de endereço de memória estão em binário; outros valores em hexadecimal.

A técnica de mapeamento direto é simples e pouco dispendiosa para se implementar. Sua principal desvantagem é que existe um local de cache fixo para cada bloco. Assim, se um programa referenciar palavras repetidamente de dois blocos diferentes, mapeados para a mesma linha, então os blocos serão continuamente trocados na cache, e a razão de acerto será baixa (um fenômeno conhecido como *thrashing*).



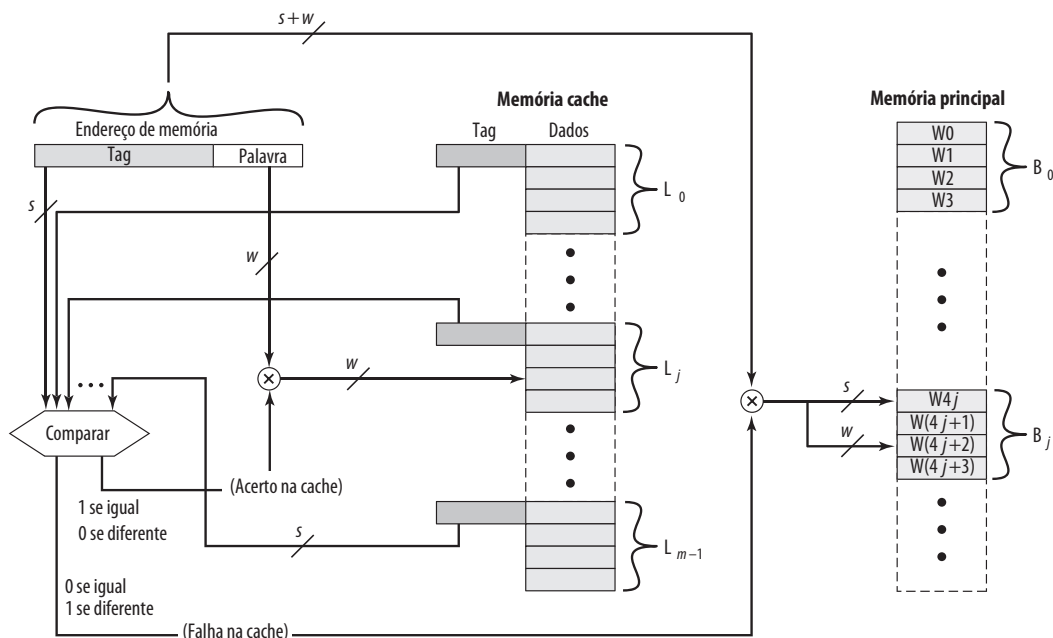
### Simulador de cache vítima seletiva

Uma técnica para diminuir a penalidade de falha é guardar o que foi descartado caso seja necessário novamente. Como os dados descartados já foram lidos, podem ser usados novamente a um custo pequeno. Essa reciclagem é possível usando uma *victim cache*. A *victim cache* foi proposta originalmente como um método para reduzir as perdas de conflito das caches mapeadas diretamente sem afetar seu tempo de acesso. A *victim cache* é uma cache totalmente associativa, cujo tamanho normalmente é de 4 a 16 linhas de cache, residindo entre uma cache L1 mapeada diretamente e o próximo nível de memória. Esse conceito é explorado no Apêndice D.

**MAPEAMENTO ASSOCIATIVO** O mapeamento associativo compensa a desvantagem do mapeamento direto, permitindo que cada bloco da memória principal seja carregado em qualquer linha da cache (Figura 4.8b). Nesse caso, a lógica de controle da cache interpreta um endereço de memória simplesmente como um campo Tag e um campo Palavra. O campo Tag identifica o bloco da memória principal. Para determinar se um bloco está na cache, a lógica de controle da cache precisa comparar simultaneamente a tag de cada linha. A Figura 4.11 ilustra a lógica. Observe que nenhum campo no endereço corresponde ao número de linha, de modo que o número de linhas na cache não é determinado pelo formato do endereço. Resumindo,

- Tamanho do endereço =  $(s + w)$  bits.
- Número de unidades endereçáveis =  $2^{s+w}$  palavras ou bytes.
- Tamanho do bloco = tamanho da linha =  $2^w$  palavras ou bytes.
- Número de blocos na memória principal =  $\frac{2^{s+w}}{2^w} = 2^s$ .

**Figura 4.11** Organização da memória cache totalmente associativa



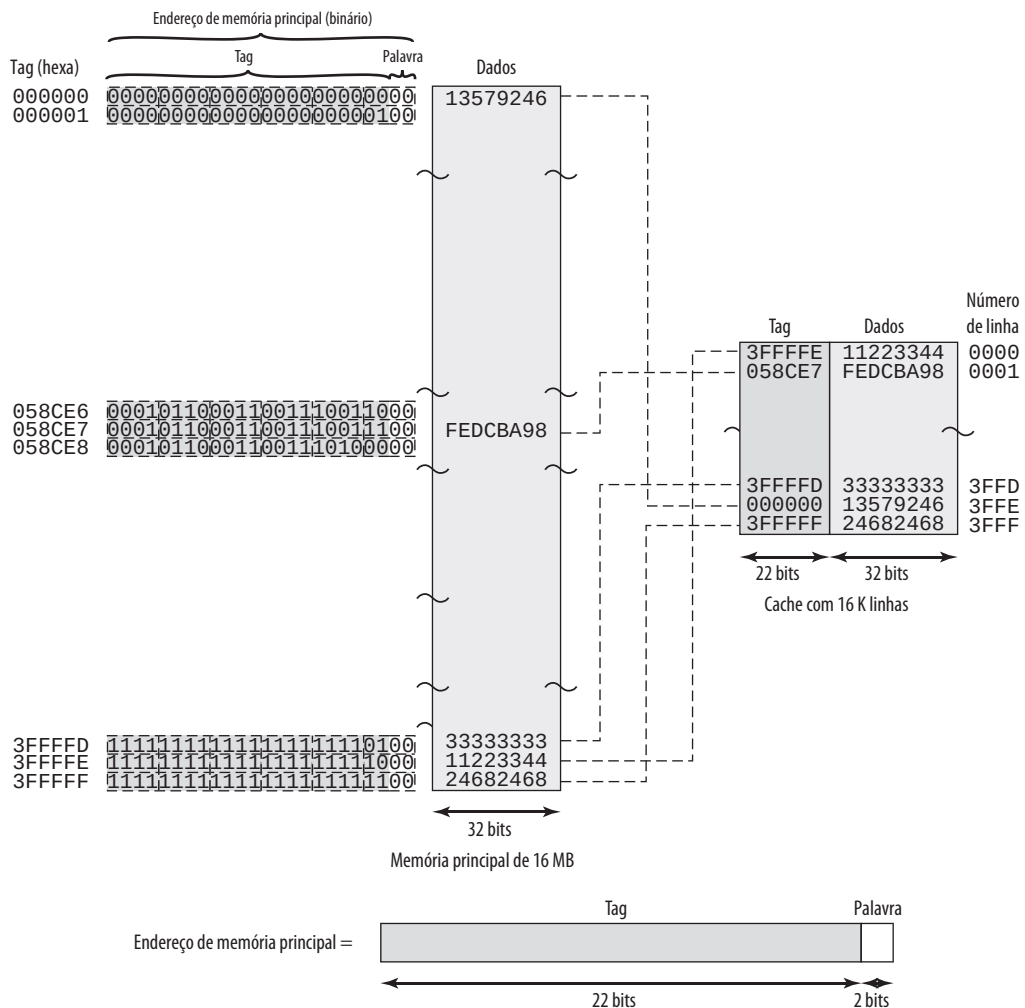


- Número de linhas na cache = indeterminado.
- Tamanho da tag =  $s$  bits.

**Exemplo 4.2b** A Figura 4.12 mostra nosso exemplo usando o mapeamento associativo. Um endereço da memória principal consiste em uma tag de 22 bits e um número do byte de 2 bits. A tag de 22 bits precisa ser armazenada com o bloco de dados de 32 bits para cada linha na cache. Observe que são os 22 bits mais à esquerda (mais significativos) do endereço que formam a tag. Assim, o endereço hexadecimal de 24 bits 16339C tem a tag de 22 bits 058CE7. Isso pode ser visto facilmente na notação binária:

|                               |      |      |      |      |      |      |           |
|-------------------------------|------|------|------|------|------|------|-----------|
| endereço de memória           | 0001 | 0110 | 0011 | 0011 | 1001 | 1100 | (binário) |
|                               | 1    | 6    | 3    | 3    | 9    | C    | (hexa)    |
| tag (22 bits mais à esquerda) | 00   | 0101 | 1000 | 1100 | 1110 | 0111 | (binário) |
|                               | 0    | 5    | 8    | C    | E    | 7    | (hexa)    |

**Figura 4.12** Exemplo de mapeamento associativo



Nota: valores de endereço de memória em binário; outros, em hexadecimal

Com o mapeamento associativo, existe flexibilidade em relação a qual bloco substituir quando um novo bloco for lido para a cache. Os algoritmos de substituição, discutidos mais adiante nesta seção, são projetados para maximizar a razão de acerto. A principal desvantagem do mapeamento associativo é a complexidade do circuito necessário para comparar as **tags** de todas as linhas da cache em paralelo.



### Simulador de análise de tempo de cache

**MAPEAMENTO ASSOCIATIVO EM CONJUNTO (SET ASSOCIATIVE)** O mapeamento associativo em conjunto é um meio-termo que realça os pontos fortes das técnicas direta e associativa, enquanto reduz suas desvantagens.

Neste caso, a cache é uma série de conjuntos, cada um consistindo em uma série de linhas. Os relacionamentos são:

$$m = v \times k$$

$$i = j \text{ módulo } v$$

onde

$i$  = número do conjunto de cache

$j$  = número de bloco da memória principal

$m$  = número de linhas na cache

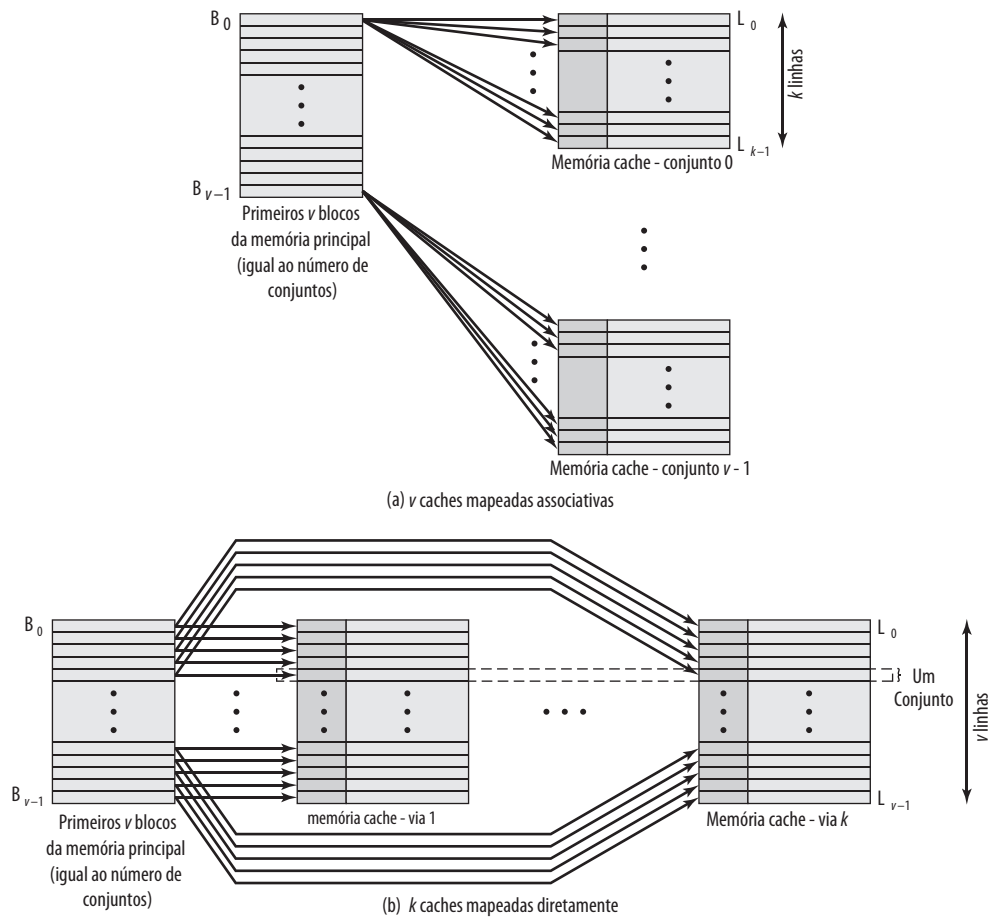
$v$  = número de conjuntos

$k$  = número de linhas em cada conjunto

Isso é conhecido como mapeamento associativo em conjunto com  $k$  linhas por conjunto ( $k$ -way). Com o mapeamento associativo em conjunto, o bloco  $B_j$  pode ser mapeado para qualquer uma das linhas do conjunto  $j$ . A Figura 4.13a ilustra esse mapeamento para os primeiros  $v$  blocos da memória principal. Assim como no mapeamento associativo, cada palavra é mapeada para múltiplas linhas de cache. Para o mapeamento associativo em conjunto, cada palavra é mapeada para todas as linhas de cache em um conjunto específico, de modo que o bloco  $B_0$  da memória principal é mapeada no conjunto 0, e assim por diante. Assim, a cache associativa em conjunto pode ser implementada fisicamente como  $v$  caches associativas. Também é possível implementar a cache associativa em conjunto como  $k$  caches de mapeamento direto, como mostra a Figura 4.13b. Cada cache mapeada diretamente é conhecida como uma via, consistindo em  $v$  linhas. As primeiras  $v$  linhas da memória principal são mapeadas diretamente nas  $v$  linhas de cada via; o próximo grupo de  $v$  linhas da memória principal é mapeado de modo semelhante, e assim por diante. A implementação mapeada diretamente em geral é usada para pequenos graus de associatividade (valores pequenos de  $k$ ) enquanto a implementação com mapeamento associativo normalmente é usada para graus de associatividade mais altos (JACOB, NG E WANG, 2008<sup>h</sup>).

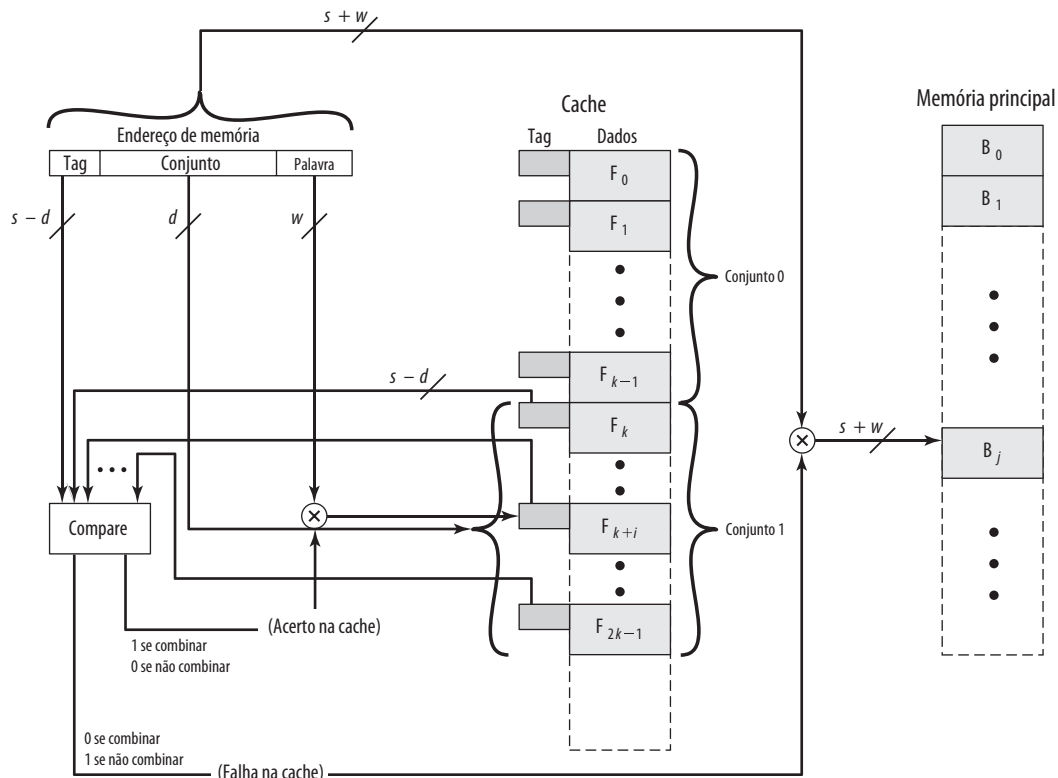
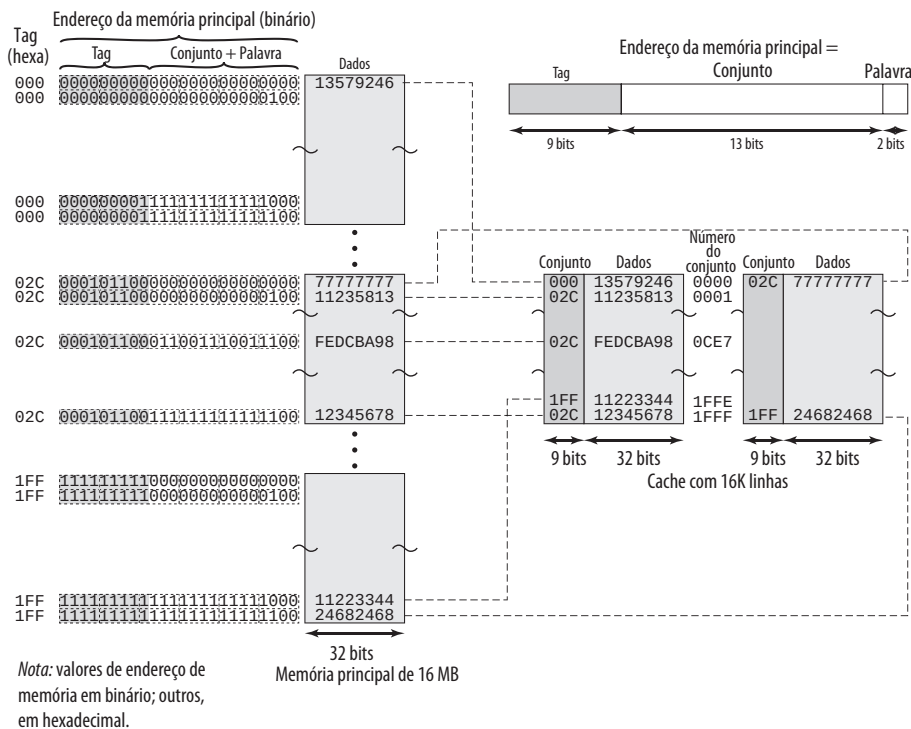
Para o mapeamento associativo em conjunto, a lógica de controle de cache interpreta um endereço de memória como três campos: Tag, Set e Palavra. Os  $d$  bits especificam um dos  $v = 2^d$  conjuntos. Os  $s$  bits dos campos Tag e Set especificam um dos  $2^s$  blocos da memória principal. A Figura 4.14 ilustra a lógica de controle de cache. Com o mapeamento totalmente associativo, a tag em um endereço de memória é muito grande e precisa ser comparada com a tag de cada linha na cache. Com o mapeamento associativo em conjunto com  $k$  vias, a tag em um endereço de memória é muito menor e só é comparada com as  $k$  tags dentro de um único conjunto. Resumindo,

- Tamanho do endereço =  $(s + w)$  bits.
- Número de unidades endereçáveis =  $2^{s+w}$  palavras ou bytes.
- Tamanho do bloco = tamanho da linha =  $2^w$  palavras ou bytes.
- Número de blocos na memória principal =  $\frac{2^{s+w}}{2^w} = 2^s$ .
- Número de linhas no conjunto =  $k$ .
- Número de conjuntos =  $v = 2^d$ .
- Número de linhas na cache =  $m = kv = k \times 2^d$ .
- Tamanho da cache =  $k \times 2^{d+w}$  palavras ou bytes.
- Tamanho da tag =  $(s - d)$  bits.

**Figura 4.13** Mapeamento da memória principal na memória cache: associativa em conjunto com  $k$  linhas por conjunto ( $k$ -way)

**Exemplo 4.2c** A Figura 4.15 mostra nosso exemplo usando o mapeamento associativo em conjunto com duas linhas em cada conjunto, denominado associativo em conjunto com duas linhas por conjunto (2-way). O número de conjunto com 13 bits identifica um conjunto exclusivo de duas linhas dentro da cache. Ele também oferece o número do bloco na memória principal, módulo  $2^{13}$ . Isso determina o mapeamento dos blocos nas linhas. Assim, os blocos 000000, 008000, ..., FF8000 da memória principal são mapeados no conjunto 0 da cache. Qualquer um desses blocos pode ser carregado em qualquer uma das duas linhas no conjunto. Observe que nenhum dos dois blocos mapeados no mesmo conjunto de cache possui o mesmo número de tag. Para uma operação de leitura, o número de 13 bits é usado para determinar qual conjunto de duas linhas deve ser examinado. As duas linhas no conjunto são examinadas comparando com o número de tag do endereço a ser acessado.

No caso extremo de  $v = m, k = 1$ , a técnica associativa em conjunto se reduz ao mapeamento direto, e para  $v = 1, k = m$ , ela se reduz ao mapeamento associativo. O uso de duas linhas por conjunto ( $v = m/2, k = 2$ ) é a organização associativa em conjunto mais comum. Ela melhora significativamente a razão de acerto em relação ao mapeamento direto. A associação em conjunto com quatro linhas por conjunto ( $v = m/4, k = 4$ ) cria uma melhoria adicional modesta por um custo adicional relativamente pequeno (Mayberry e Efland, 1984,<sup>1</sup> Hill, 1989). Outros aumentos no número de linhas por conjunto têm pouco efeito.

**Figura 4.14** Organização da memória cache associativa em conjunto com  $k$  linhas por conjunto**Figura 4.15** Exemplo de mapeamento associativo em conjunto com duas linhas por conjunto

A Figura 4.16 mostra os resultados de um estudo de simulação do desempenho da cache associativa em conjunto como uma função do tamanho da cache (Genu, 2004<sup>b</sup>). A diferença no desempenho entre mapeamento direto e associativo em conjunto com 2 linhas por conjunto é significativa até pelo menos um tamanho de cache de 64 KB. Observe também que a diferença entre duas linhas por conjunto e quatro vias a 4 KB é muito menor do que a diferença ao passar de 4 KB para 8 KB no tamanho da cache. A complexidade da cache aumenta em proporção com a associatividade e, nesse caso, não seria justificável contra o aumento no tamanho da cache para 8 ou mesmo 16 KBytes. Um ponto final a ser observado é que, além de cerca de 32 KB, o aumento no tamanho da cache não ocasiona aumento significativo no desempenho.

Os resultados da Figura 4.16 são baseados na simulação da execução de um compilador GCC. Diferentes aplicações podem gerar resultados diferentes. Por exemplo, Cantin e Hill (2001<sup>1</sup>) relatam os resultados para o desempenho da cache usando muitos dos benchmarks SPEC CPU2000. Os resultados de Cantin e Hill (2001<sup>1</sup>) na comparação da razão de acerto com o tamanho da cache seguem o mesmo padrão da Figura 4.16, mas os valores específicos são um tanto diferentes.



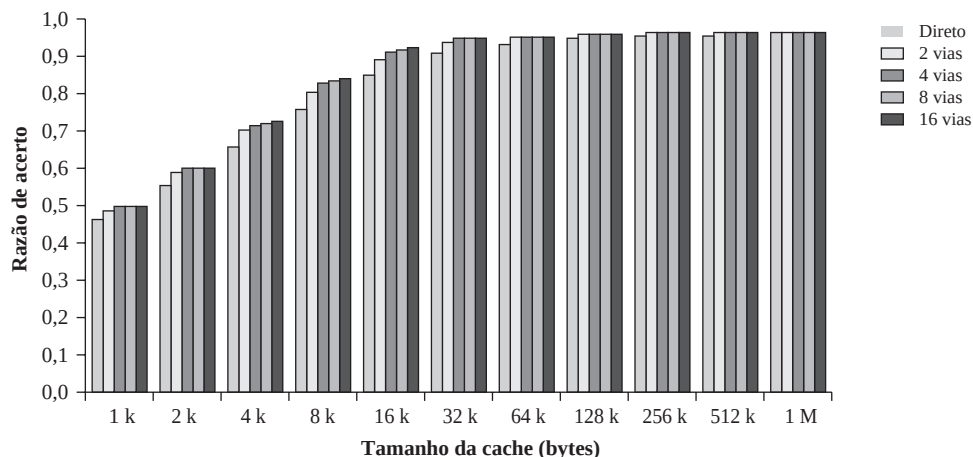
**Simulador de cache**  
**Simulador de cache multitarefa**



## Algoritmos de substituição

Uma vez que a cache estiver cheia, e um novo bloco for trazido para a cache, um dos blocos existentes precisa ser substituído. Para o mapeamento direto, existe apenas uma linha possível para qualquer bloco em particular e nenhuma escolha é possível. Para as técnicas associativa e associativa em conjunto, um algoritmo de substituição é necessário. Para alcançar alta velocidade, tal algoritmo precisa ser implementado em hardware. Diversos algoritmos foram experimentados. Mencionamos quatro dos mais comuns. Provavelmente, o mais eficaz seja o usado menos recentemente (LRU, do inglês *least recently used*): substitua aquele bloco no conjunto que permaneceu na cache por mais tempo sem qualquer referência a ele. Para a associatividade em conjunto com duas linhas por conjunto, isso é facilmente implementado. Cada linha inclui um bit USE. Quando uma linha é referenciada, seu bit USE é definido como 1, e o bit USE da outra linha nesse conjunto é definido como 0. Quando um bloco tiver que ser lido para o conjunto, a linha cujo bit USE for 0 é utilizada. Como estamos supondo que os locais de memória usados mais recentemente são mais prováveis de serem referenciados, LRU deverá oferecer a melhor razão de acerto. LRU também é relativamente fácil de implementar para uma cache totalmente associativa. O mecanismo de cache mantém uma lista separada de índices para todas as linhas na cache. Quando uma linha é referenciada, ela passa para a

**Figura 4.16** Associatividade variável pelo tamanho da cache



frente da lista. Para substituição, a linha no final da lista é usada. Devido à sua simplicidade de implementação, LRU é o algoritmo de substituição mais popular.

Outra possibilidade é primeiro a entrar, primeiro a sair (FIFO, do inglês *first-in-first-out*): substitua o bloco no conjunto que esteve na cache por mais tempo. O algoritmo FIFO é facilmente implementado como uma técnica round-robin ou de buffer circular. Ainda outra possibilidade de algoritmo é o usado menos frequentemente (LFU, do inglês *least frequently used*): substitua aquele bloco no conjunto que teve menos referências. O algoritmo LFU poderia ser implementado associando um contador a cada linha. Uma técnica não baseada no uso (ou seja, não LRU, LFU, FIFO ou alguma variante) é escolher uma linha aleatória dentre as linhas candidatas. Estudos de simulação têm mostrado que a substituição aleatória oferece um desempenho apenas ligeiramente inferior a um algoritmo baseado no uso (Smith, 1982<sup>m</sup>).



## Política de escrita

Quando um bloco que está residente na cache estiver para ser substituído, existem dois casos a considerar. Se o bloco antigo na cache não tiver sido alterado, então ele pode ser substituído por um novo bloco sem primeiro atualizar o bloco antigo. Se pelo menos uma operação de escrita tiver sido realizada em uma palavra nessa linha da cache, então a memória principal precisa ser atualizada escrevendo a linha de cache no bloco de memória antes de trazer o novo bloco. Diversas políticas de escrita são possíveis, com escolhas econômicas de desempenho. Existem dois problemas a combater. Primeiro, mais de um dispositivo pode ter acesso à memória principal. Por exemplo, um módulo de E/S pode ser capaz de ler-escrever diretamente na memória. Se uma palavra tiver sido alterada apenas na cache, então a palavra correspondente da memória é inválida. Além do mais, se o dispositivo de E/S tiver alterado a memória principal, então a palavra da cache é inválida. Um problema mais complexo ocorre quando múltiplos processadores são conectados ao mesmo barramento e cada processador tem sua própria cache local. Então, se uma palavra for alterada em uma cache, ela possivelmente poderia invalidar uma palavra em outras caches.

A técnica mais simples é denominada **write-through**. Usando essa técnica, todas as operações de escrita são feitas na memória principal e também na cache, garantindo que a memória principal sempre seja válida. Qualquer outro módulo processador-cache pode monitorar o tráfego para a memória principal para manter a consistência dentro de sua própria cache. A principal desvantagem dessa técnica é que ela gera um tráfego de memória considerável e podendo vir a ser um gargalo. Uma técnica alternativa, conhecida como **write-back**, minimiza as escritas na memória. Com **write-back**, as atualizações são feitas apenas na cache. Quando ocorre uma atualização, um **bit de modificação**, ou **bit de uso**, associado à linha, é marcado. Depois, quando um bloco é substituído, ele é escrito de volta na memória principal se, e somente se, o bit de modificação estiver marcado. O problema com **write-back** é que partes da memória principal podem ficar inválidas, e daí os acessos pelos módulos de E/S só podem ser permitidos pela cache. Isso exige circuitos complexos e gera um gargalo em potencial. A experiência tem mostrado que a percentagem de referências à memória que são escritas está na ordem de 15% (SMITH, 1982<sup>m</sup>). Porém, para aplicações de HPC, esse número pode se aproximar a 33% (multiplicação de vetores), e pode chegar a até 50% (transposição de matrizes).

**Exemplo 4.3** Considere uma cache com um tamanho de linha de 32 bytes e uma memória principal que requer 30 ns para transferir uma palavra de 4 bytes para qualquer linha que seja escrita pelo menos uma vez antes de ser retirada da cache; qual é o número médio de vezes que a linha precisa ser escrita antes de ser retirada para que uma cache **write-back** seja mais eficiente do que uma cache **write-through**?

Para o caso **write-back**, cada linha modificada é escrita de volta uma vez, no momento da troca, usando  $8 \times 30 = 240$  ns. Para o caso **write-through**, cada atualização da linha requer que uma palavra seja escrita na memória principal, usando 30 ns. Portanto, se a linha média que é escrita pelo menos uma vez for escrita mais de 8 vezes antes de ser trocada, então **write-back** é mais eficiente.

Em uma organização de barramento em que mais de um dispositivo (normalmente, um processador) tem uma cache e a memória principal é compartilhada, um novo problema é introduzido. Se os dados em uma cache forem alterados, isso invalida não apenas a palavra correspondente na memória principal, mas também essa mesma

palavra em outras caches (se qualquer outra cache tiver essa mesma palavra). Mesmo que uma política *write-through* seja usada, as outras caches podem conter dados inválidos. Diz-se que um sistema que impede esse problema mantém coerência de cache. Algumas das técnicas possíveis para a coerência de cache são:

- **Observação do barramento com *write-through*:** cada controlador de cache monitora as linhas de endereço para detectar as operações de escrita para a memória por outros mestres de barramento. Se outro mestre escrever em um local na memória compartilhada que também reside na memória cache, o controlador de cache invalida essa entrada da cache. Essa estratégia depende do uso de uma política *write-through* por todos os controladores de cache.
- **Transparência do hardware:** um hardware adicional é usado para garantir que todas as atualizações na memória principal por meio da cache sejam refletidas em todas as caches. Assim, se um processador modificar uma palavra em sua cache, essa atualização é escrita na memória principal. Além disso, quaisquer palavras correspondentes em outras caches são semelhantemente atualizadas.
- **Memória não chacheável:** somente uma parte da memória principal é compartilhada por mais de um processador, e esta é designada como não mantida em cache. Nesse tipo de sistema, todos os acessos à memória compartilhada são falhas de cache, pois a memória compartilhada nunca é copiada para a cache. A memória não mantida em cache pode ser identificada usando lógica de seleção de chip ou bits mais significativos de endereço alto.

A coerência de cache é um campo de pesquisa atual. Esse assunto é explorado com mais detalhes na Parte 5.



## Tamanho da linha

Outro elemento de projeto é o tamanho da linha. Quando um bloco de dados é recuperado e colocado na cache, não apenas a palavra desejada, mas também algumas palavras adjacentes são armazenadas. À medida que o tamanho do bloco aumenta de tamanhos muito pequenos para maiores, a razão de acerto a princípio aumentará devido ao princípio da localidade, que diz que os dados nas vizinhanças de uma palavra referenciada provavelmente serão referenciados no futuro próximo. À medida que o tamanho do bloco aumenta, dados mais úteis são trazidos para a cache. Contudo, a razão de acerto começará a diminuir enquanto o bloco se torna ainda maior e a probabilidade de uso da informação recém-trazida se torna menor que a probabilidade de reutilizar as informações que foram substituídas. Dois efeitos específicos entram em cena:

- Blocos maiores reduzem o número de blocos que cabem em uma cache. Como cada busca de bloco escreve sobre o conteúdo antigo da cache, um número pequeno de blocos resulta em dados sendo modificados pouco depois de serem buscados.
- À medida que o bloco se torna maior, cada palavra adicional fica mais distante da palavra solicitada e, portanto, tem menos probabilidade de ser necessária no futuro próximo.

O relacionamento entre o tamanho do bloco e a razão de acerto é complexo, dependendo das características de localidade de um programa em particular, e nenhum valor ideal definitivo foi encontrado. Um tamanho de 8 a 64 bytes parece ser razoavelmente próximo do ideal (SMITH, 1987<sup>n</sup>; PRZYBYLSKI, HOROWITZ e HENNESSY, 1988<sup>o</sup>; PRZYBYLSKI, 1990<sup>o</sup>; HANDY, 1993<sup>a</sup>). Para sistemas HPC, tamanhos de linha de cache com 64 e 128 bytes são usados com mais frequência.



## Número de memórias caches

Quando as memórias caches foram introduzidas originalmente, o sistema de memória típico tinha uma única cache. Mais recentemente, o uso de múltiplas caches tem se tornado comum. Dois aspectos de projeto dizem respeito ao número de níveis de memórias caches e ao uso de caches unificadas ou separadas.

**CACHES MULTINÍVEL** À medida que a densidade lógica aumenta, torna-se possível ter uma cache no mesmo chip que o processador: a cache no chip (*on chip*). Em comparação com uma cache conectada por meio de um barramento externo, a cache no chip reduz a atividade do barramento externo do processador e, portanto, agiliza o tempo de execução e aumenta o desempenho geral do sistema. Quando a instrução a leitura dos dados são feitos na cache no chip, não existe o acesso ao barramento. Com os caminhos de da-

dos internos ao processador são curtos, em comparação com o tamanho do barramento, os acessos à cache no chip serão feitos mais rápido que os ciclos de barramento com estado *zero-wait* (tempo de espera nulo). Além do mais, durante esse período, o barramento estará livre para aceitar outras transferências.

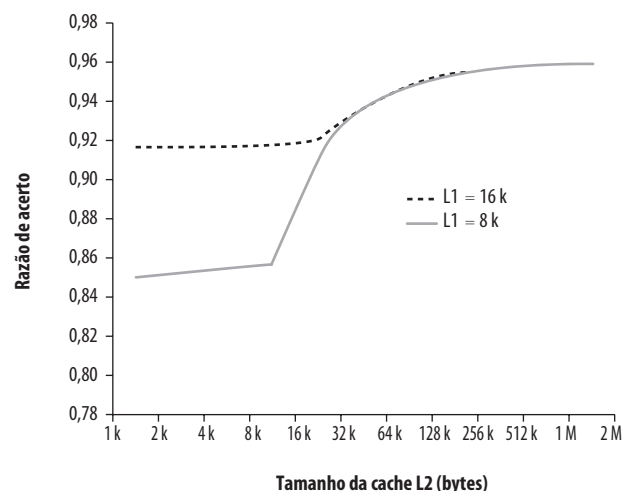
A inclusão de uma cache no chip deixa aberta a questão de se uma cache fora do chip, ou externa, ainda é desejável. Normalmente, a resposta é sim, e a maior parte dos projetos contemporâneos inclui caches dentro e fora do chip. A organização mais simples desse tipo é conhecida como uma cache de dois níveis, com a cache interna designada como nível 1 (L1 — *level 1*) e a cache externa designada como nível 2 (L2). O motivo para incluir uma cache L2 é o seguinte: se não houver cache L2 e o processador fizer uma solicitação de acesso para um local de memória que não esteja na cache L1, então o processador precisa acessar a memória DRAM ou ROM pelo barramento. Devido à baixa velocidade do barramento e ao alto tempo de acesso à memória, tem-se um desempenho. Por outro lado, se uma cache L2 SRAM (RAM estática) for usada, então normalmente a informação que falta pode ser recuperada rapidamente. Se a SRAM for rápida o suficiente para corresponder à velocidade do barramento, então os dados podem ser acessados usando uma transação no estado *zero-wait*, o tipo mais rápido de transferência de barramento.

Dois recursos de projeto moderno de cache para caches multinível merecem ser citados. Primeiro, para uma cache L2 fora do chip, muitos projetos não usam o barramento do sistema como caminho para transferência entre a cache L2 e o processador, mas usam um caminho de dados separado, a fim de reduzir a carga sobre o barramento do sistema. Segundo, com o encolhimento contínuo dos componentes do processador, diversos processadores agora incorporam a cache L2 no chip do processador, melhorando o desempenho.

A economia em potencial devido ao uso de uma cache L2 depende da taxas de acerto nas caches L1 e L2. Vários estudos têm mostrado que, em geral, o uso de uma cache de segundo nível melhora o desempenho (por exemplo, ver Azimi, Prosod e Bhat (1992<sup>3</sup>); Novitsky, Azimi e Ghaznavi (1993<sup>3</sup>); Handy (1993<sup>4</sup>). Porém, o uso de caches multinível complica todas as questões de projeto relacionadas a caches, incluindo tamanho, algoritmo de substituição e política de escrita; veja algumas discussões a respeito disso em Handy (1993) e Peir, Hsu e Smith (1999<sup>5</sup>).

A Figura 4.17 mostra os resultados de um estudo de simulação de desempenho da cache de dois níveis como uma função do tamanho da cache (GENU, 2004<sup>6</sup>). A figura pressupõe que as duas caches têm o mesmo tamanho de linha e mostra a razão de acerto total. Ou seja, um acerto é contado se os dados desejados aparecerem na cache L1 ou L2. A figura mostra o impacto da L2 sobre os acertos totais com relação ao tamanho da L1. L2 tem pouco efeito sobre o número total de acertos de cache até que seja pelo menos o dobro do tamanho da

**Figura 4.17** Razão de acerto total (L1 e L2) para L1 de 8 KB e 16 KB





cache L1. Observe que a parte mais íngreme da inclinação para uma cache L1 de 8 KBytes é para uma cache L2 de 16 KBytes. Novamente para uma cache L1 de 16 KBytes, a parte mais íngreme da curva é para um tamanho de cache L2 de 32 KBytes. Antes desse ponto, a cache L2 tem pouco ou nenhum impacto sobre o desempenho da cache total. A necessidade de a cache L2 ser maior que a cache L1 para afetar o desempenho faz sentido. Se a cache L2 tiver o mesmo tamanho de linha e capacidade da cache L1, seu conteúdo mais ou menos espelhará o da cache L1.

Com a disponibilidade cada vez maior de área no chip, a maior parte dos microprocessadores modernos passou a cache L2 para dentro do chip processador e acrescentou uma cache L3. Originalmente, a cache L3 era acessível pelo barramento externo. Mais recentemente, a maioria dos microprocessadores incorporou uma cache L3 no chip. De qualquer forma, parece haver uma vantagem no desempenho para acrescentar um terceiro nível (por exemplo, ver GHAI, JOYNER e JOHN, 1998°).

**CACHES UNIFICADAS VERSUS SEPARADAS** Quando a cache no chip apareceu inicialmente, muitos dos projetos consistiam em uma única cache usada para armazenar referências a dados e instruções. Mais recentemente, tornou-se comum dividir a cache em duas: uma dedicada a instruções e uma dedicada a dados. Essas duas caches existem no mesmo nível, normalmente como duas caches L1. Quando o processador tenta buscar uma instrução da memória principal, ele primeiro consulta a cache L1 de instrução, e quando o processador tenta buscar dados da memória principal, ele primeiro consulta a cache L1 de dados.

Existem duas vantagens em potencial de uma cache unificada:

- Para determinado tamanho de cache, uma cache unificada tem uma taxa de acerto mais alta que as caches divididas, pois ela equilibra a carga entre buscas de instrução e dados automaticamente. Ou seja, se um padrão de execução envolve muito mais buscas de instrução do que buscas de dados, então a cache tenderá a ser preenchida com instruções, e se um padrão de execução envolve relativamente mais buscas de dados, acontecerá o oposto.
- Somente uma cache precisa ser projetada e implementada.

Apesar dessas vantagens, a tendência é em direção a caches separadas, particularmente para máquina superesca- lares, como o Pentium e o PowerPC, que enfatizam a execução de instrução paralela e a pré-busca de instruções futuras previsíveis. A principal vantagem do projeto de cache separada é que isso elimina a disputa pela cache entre a unidade de busca/decodificação de instrução e a unidade de execução. Isso é importante em qualquer projeto que conta com o pipeline de instruções. Normalmente, o processador buscará instruções antes da hora e preencherá um buffer, ou pipeline, com instruções a serem executadas. Suponha agora que tenhamos uma cache de instrução/dados unificada. Quando a unidade de execução realiza um acesso à memória para carregar e armazenar dados, a solicitação é submetida à cache unificada. Se, ao mesmo tempo, o mecanismo de pré-busca de instrução emitir uma solicitação de leitura à cache para uma instrução, essa solicitação será temporariamente bloqueada para que a cache possa atender a unidade de execução primeiro, permitindo que ela complete a instrução atualmente em execução. Essa disputa pela cache pode diminuir o desempenho, interferindo com o uso eficiente da pipeline de instruções. A estrutura de cache separada contorna essa dificuldade.



## 4.4 Organização da memória cache do Pentium 4

A evolução da organização da cache é vista claramente na evolução dos microprocessadores Intel (Tabela 4.4). O 80386 não inclui uma cache no chip. O 80486 inclui uma única cache no chip de 8 KBytes, usando um tamanho de linha de 16 bytes e uma organização associativa em conjunto com quatro linhas por conjunto. Todos os processadores Pentium incluem duas caches L1 no chip, uma para dados e uma para instruções. Para o Pentium 4, a cache de dados L1 tem 16 KBytes, usando um tamanho de linha de 64 bytes e uma organização associativa em conjunto com quatro linhas por conjunto. A cache de instruções do Pentium 4 é descrita mais adiante. O Pentium II também inclui uma cache L2 que alimenta ambas as caches L1. A cache L2 é associativa em conjunto com oito linhas por conjuntos, com um tamanho de 512 KB e um tamanho de linha de 128 bytes. Uma cache L3 foi acrescentada para o Pentium III, e passou a residir no chip com as versões avançadas do Pentium 4.

**Tabela 4.4** Evolução de cache da Intel

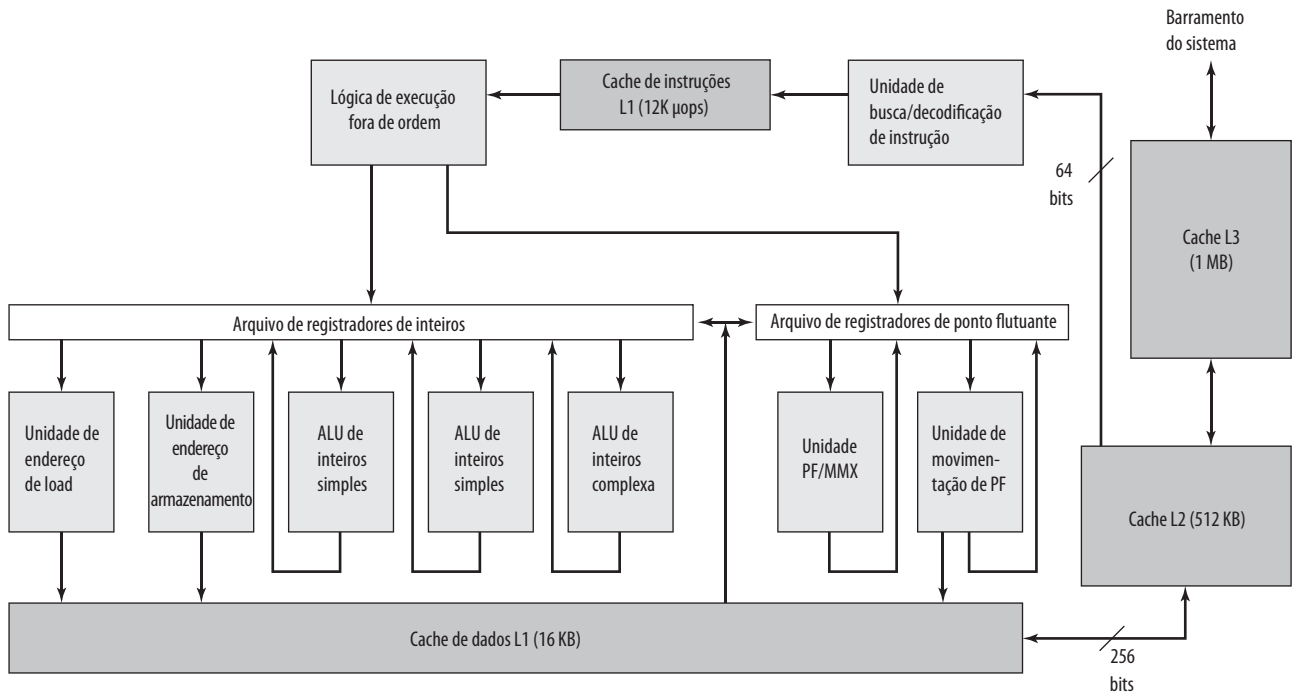
| Problema                                                                                                                                                                                                                               | Solução                                                                                                                                                                   | Processador em que o recurso apareceu inicialmente |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------|
| Memória externa mais lenta que o barramento do sistema.                                                                                                                                                                                | Acrescentar cache externa usando tecnologia de memória mais rápida                                                                                                        | 386                                                |
| O aumento da velocidade do processador torna o barramento externo um gargalo para o acesso à memória cache..                                                                                                                           | Mover a cache externa para o chip, trabalhando na mesma velocidade do processador                                                                                         | 486                                                |
| Cache interna um tanto pequena, devido ao espaço limitado no chip.                                                                                                                                                                     | Acrescentar cache L2 externa usando tecnologia mais rápida que a memória principal                                                                                        | 486                                                |
| Quando ocorre uma disputa entre o mecanismo de pré-busca de instruções e a unidade de execução no acesso simultâneo à memória cache. Nesse caso, a busca antecipada é adiada até o término do acesso da unidade de execução aos dados. | Criar caches separadas para dados e instruções                                                                                                                            | Pentium                                            |
| Maior velocidade do processador torna o barramento externo um gargalo para o acesso à cache L2.                                                                                                                                        | Criar barramento back-side separado, que trabalha com velocidade mais alta que o barramento externo principal (front-side). O barramento back-side é dedicado à cache L2. | Pentium Pro                                        |
|                                                                                                                                                                                                                                        | Mover cache L2 para o chip do processador.                                                                                                                                | Pentium II                                         |
| Algumas aplicações lidam com bancos de dados enormes, e precisam ter acesso rápido a grandes quantidades de dados. As caches no chip são muito pequenas.                                                                               | Acrescentar cache L3 externa.                                                                                                                                             | Pentium III                                        |
|                                                                                                                                                                                                                                        | Mover cache L3 para o chip.                                                                                                                                               | Pentium 4                                          |

A Figura 4.18 contém uma visão simplificada da organização do Pentium 4, destacando o posicionamento das três caches. O núcleo do processador consiste em quatro componentes principais:

- **Unidade de busca/decodificação:** busca instruções do programa na ordem a partir da cache L2, decodifica-as para uma série de micro-operações e armazena os resultados na cache de instruções L1.
- **Lógica de execução fora da ordem:** escalona a execução das micro-operações, sujeito a dependências de dados e disponibilidade de recursos; assim, as micro-operações podem ser escalonadas para execução em uma ordem diferente daquela em que foram obtidas do fluxo de instruções. Se o tempo permitir, essa unidade escalona a execução antecipadamente micro-operações que podem ser solicitadas no futuro.
- **Unidades de execução:** essas unidades executam micro-operações, buscando os dados solicitados da cache de dados L1 e armazenando os resultados temporariamente em registradores.
- **Subsistema de memória:** essa unidade inclui as caches L2 e L3 e o barramento do sistema, que é usado para acessar a memória principal quando as caches L1 e L2 tiverem uma falta de cache e para acessar os recursos de E/S do sistema.

Diferente da organização usada em todos os modelos Pentium anteriores, e na maioria dos outros processadores, a cache de instruções do Pentium 4 localiza-se entre a lógica de decodificação de instrução e o núcleo de execução. O raciocínio por trás dessa decisão de projeto é o seguinte: conforme discutiremos com mais detalhes no Capítulo 14, o processo do Pentium decodifica, ou traduz, instruções de máquina do Pentium para instruções simples tipo RISC, chamadas de micro-operações. O uso de micro-operações simples, de tamanho fixo, permite o uso do pipeline superescalar e técnicas de escalonamento que melhoram o desempenho. Contudo, as instruções de máquina do Pentium são difíceis de decodificar; elas têm um número variável de bytes e muitas opções diferentes. Acontece que o desempenho é melhorado se essa decodificação for feita independentemente da lógica de escalonamento e pipeline. Retornaremos a esse tópico no Capítulo 14.

**Figura 4.18** Diagrama em blocos do Pentium 4



A cache de dados emprega uma política *write-back*: os dados são escritos na memória principal apenas quando são removidos da cache e houver uma atualização. O processador Pentium 4 pode ser configurado dinamicamente para aceitar a política *write-through*.

A cache de dados L1 é controlada por dois bits em um dos registradores de controle, rotulados como bits CD (*cache disable*) e NW (*not write-through*) (Tabela 4.5). Há também duas instruções do Pentium 4 que podem ser usadas para controlar a cache de dados: INVD invalida (esvazia) a memória cache interna e sinaliza a cache externa (se houver) para invalidar. WBINVD escreve de volta e invalida a cache interna e depois escreve de volta e invalida a cache externa.

As caches L2 e L3 são associativas em conjunto com oito linhas por conjunto, com um tamanho de linha de 128 bytes.



## 4.5 Organização de cache da ARM

A organização de cache da ARM evoluiu com a arquitetura geral da família ARM, refletindo a busca contínua de desempenho, que é a força motriz para todos os projetistas de microprocessador.

**Tabela 4.5** Modos operacionais da cache do Pentium 4

| Bits de controle |    | Modo operacional |                |              |
|------------------|----|------------------|----------------|--------------|
| CD               | NW | Cache Fills      | Write-throughs | Invalidates  |
| 0                | 0  | Habilitado       | Habilitado     | Habilitado   |
| 1                | 0  | Desabilitado     | Habilitado     | Habilitado   |
| 1                | 1  | Desabilitado     | Desabilitado   | Desabilitado |

**Nota:** CD = 0; NW = 1 é uma combinação inválida.

A Tabela 4.6 mostra essa evolução. Os modelos ARM7 usavam uma cache L1 unificada, enquanto todos os modelos subsequentes usam uma separada dividida para instruções e dados. Todos os projetos ARM utilizam uma cache associativa em conjunto, com o grau de associatividade e o tamanho de linha variado. Os núcleos em cache do ARM com uma MMU utilizam uma cache lógica para a família de processador de ARM7 até ARM10, incluindo os processadores Intel StrongARM e Intel Xscale. A família ARM11 usa uma cache física. A diferença entre cache lógica e física já foi discutida anteriormente neste capítulo (Figura 4.7).

Um recurso interessante da arquitetura ARM é o uso de um pequeno buffer de escrita *first-in-first-out* (FIFO) para melhorar o desempenho de escrita da memória. O buffer de escrita é interposto entre a cache e a memória principal, e consiste em um conjunto de endereços e um conjunto de palavras de dados. O buffer de escrita é pequeno em comparação com a cache, e pode manter até quatro endereços independentes. Normalmente, o buffer de escrita está habilitado para toda a memória principal, embora possa ser seletivamente desabilitado no nível de página. A Figura 4.19, retirada de Sloss, Symes e Wright (2004<sup>4</sup>), mostra o relacionamento entre buffer de escrita, da cache e memória principal.

O buffer de escrita opera da seguinte forma: quando o processador realiza uma escrita em uma área armazenamento temporário, os dados são colocados no buffer de escrita na velocidade de clock do processador e o processador continua a execução. Uma escrita ocorre quando os dados na cache forem escritos de volta à memória principal. Assim, os dados a serem escritos são transferidos da cache para o buffer de escrita. O buffer de escrita, então, realiza a escrita externa em paralelo. Porém, se o buffer de escrita estiver cheio (seja porque já existe o número máximo de palavras de dados no buffer ou porque não existe slot para o novo endereço), então as operações do processador são adiadas ("stall" no processador) até que haja espaço suficiente no buffer. Enquanto as operações que não são de escrita prosseguem, o buffer de escrita continua a escrever na memória principal até que o buffer esteja completamente vazio.

Os dados escritos no buffer de escrita não estão disponíveis para leitura de volta na cache até que os dados tenham sido transferidos do buffer de escrita para a memória principal. Esse é o principal motivo para o buffer de escrita ser muito pequeno. Mesmo assim, a menos que exista uma proporção grande de escritas em um programa em execução, o buffer de escrita melhora o desempenho.



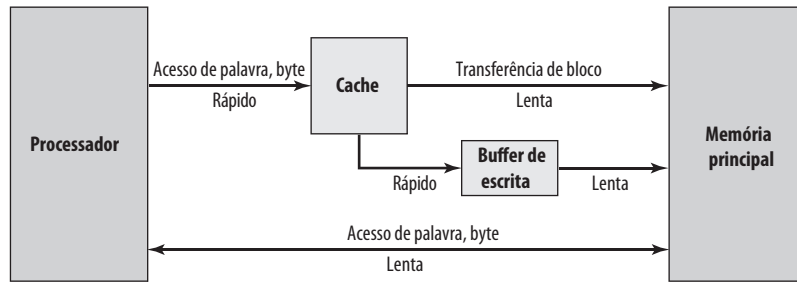
## 4.6 Leitura recomendada

Jacob, Ng e Wang (2008<sup>h</sup>) tem um tratamento excelente e atualizado sobre projeto de cache. Outro tratamento profundo é Handy (1993<sup>o</sup>). Um artigo clássico que ainda merece ser lido é Smith (1982<sup>m</sup>); ele estuda os diversos elementos do projeto de cache e apresenta os resultados de um grande conjunto de análises. Outro clássico interessante é Wilkes (1965<sup>w</sup>), que provavelmente é o primeiro artigo a introduzir o conceito da cache. Goodman (1983<sup>o</sup>)

**Tabela 4.6** Características da memória cache do ARM

| Núcleo          | Tipo de cache | Tamanho de cache (kB) | Tamanho da linha de cache (palavras) | Associatividade (linhas por conjunto) | Local  | Tamanho do buffer de escrita (palavras) |
|-----------------|---------------|-----------------------|--------------------------------------|---------------------------------------|--------|-----------------------------------------|
| ARM720T         | Unificada     | 8                     | 4                                    | 4 linhas por conjunto                 | Lógico | 8                                       |
| ARM920T         | Separada      | 16/16 D/I             | 8                                    | 64 linhas por conjunto                | Lógico | 16                                      |
| ARM926EJ-S      | Separada      | 4-128/4-128 D/I       | 8                                    | 4 linhas por conjunto                 | Lógico | 16                                      |
| ARM1022E        | Separada      | 16/16 D/I             | 8                                    | 64 linhas por conjunto                | Lógico | 16                                      |
| ARM1026EJ-S     | Separada      | 4-128/4-128 D/I       | 8                                    | 4 linhas por conjunto                 | Lógico | 8                                       |
| Intel StrongARM | Separada      | 16/16 D/I             | 4                                    | 32 linhas por conjunto                | Lógico | 32                                      |
| Intel Xscale    | Separada      | 32/32 D/I             | 8                                    | 32 linhas por conjunto                | Lógico | 32                                      |
| ARM11 36-JF-S   | Separada      | 4-64/4-64 D/I         | 8                                    | 4 linhas por conjunto                 | Físico | 32                                      |

**Figura 4.19** Organização da cache e do buffer de escrita do ARM



também fornece uma análise útil do comportamento da cache. Outra análise valiosa é Bell, Casasent e Bell (1974<sup>v</sup>). Agarwal (1989<sup>z</sup>) que apresenta um exame detalhado de uma série de questões de projeto de cache relacionadas a multiprogramação e multiprocessamento. Higbie (1990<sup>aa</sup>) oferece um conjunto de fórmulas simples que podem ser usadas para estimar o desempenho da cache como uma função de diversos parâmetros de cache.

## Principais termos, perguntas de revisão e problemas

### Principais termos

|                                      |                           |                                    |
|--------------------------------------|---------------------------|------------------------------------|
| Tempo de acesso                      | Cache de instruções       | Acesso sequencial                  |
| Mapeamento associativo               | Cache L1                  | Mapeamento associativo em conjunto |
| Acerto de cache ( <i>cache hit</i> ) | Cache L2                  | Localidade espacial                |
| Linha de cache ( <i>cache miss</i> ) | Cache L3                  | Cache separada                     |
| Memória cache                        | Localidade                | Tag                                |
| Falha de cache                       | Cache lógica              | Localidade temporal                |
| Conjunto de cache                    | Hierarquia de memória     | Cache unificada                    |
| Cache de dados                       | Cache multinível          | Cache virtual                      |
| Acesso direto                        | Cache física              | <i>Write-back</i>                  |
| Mapeamento direto                    | Acesso aleatório          | <i>Write once</i>                  |
| Computação de alto desempenho (HPC)  | Algoritmo de substituição | <i>Write-through</i>               |
| Razão de acerto                      |                           |                                    |

### Perguntas de revisão

- 4.1 Quais são as diferenças entre acesso sequencial, acesso direto e acesso aleatório?
- 4.2 Qual é o relacionamento geral entre tempo de acesso, custo de memória e capacidade?
- 4.3 Como o princípio de localidade se relaciona com o uso de múltiplos níveis de memória?
- 4.4 Quais são as diferenças entre mapeamento direto, mapeamento associativo e mapeamento associativo em conjunto?
- 4.5 Para uma cache mapeada diretamente, um endereço de memória principal é visto como consistindo em três campos. Liste e defina os três campos.

**4.6** Para uma cache associativa, um endereço de memória principal é visto como consistindo em dois campos. Liste e defina os dois campos.

**4.7** Para uma cache associativa em conjunto, um endereço da memória principal é visto como consistindo em três campos. Liste e defina os três campos.

**4.8** Qual é a diferença entre localidade espacial e localidade temporal?

**4.9** Em geral, quais são as estratégias para explorar a localidade espacial e a localidade temporal?

### Problemas

**4.1** Uma cache associativa em conjunto consiste em 64 linhas, ou slots, divididas em conjuntos de quatro linhas. A memória principal contém 4 K blocos de 128 palavras cada. Mostre o formato dos endereços da memória principal.

**4.2** Uma cache associativa em conjunto com duas linhas por conjuntos possui linhas de 16 bytes e um tamanho total de 8 KBytes. A memória principal de 64 MBytes é endereçável por byte. Mostre o formato dos endereços da memória principal.

**4.3** Para os endereços hexadecimais da memória principal 111111, 666666, BBBB, mostre a seguinte informação, em formato hexadecimal:

- Valores de tag, linha e palavra para uma cache de mapeamento direto, usando o formato da Figura 4.10.
- Valores de tag e palavra para uma cache associativa, usando o formato da Figura 4.12.
- Valores de tag, conjunto e palavra para uma cache associativa em conjunto com duas vias, usando o formato da Figura 4.15.

**4.4** Liste os seguintes valores:

- Para o exemplo de cache direta da Figura 4.10: tamanho do endereço, número de unidades endereçáveis, tamanho de bloco, número de blocos na memória principal, número de linhas na cache, tamanho da tag.
- Para o exemplo de cache associativa da Figura 4.12: tamanho do endereço, número de unidades endereçáveis, tamanho de bloco, número de blocos na memória principal, número de linhas na cache, tamanho da tag.
- Para o exemplo de cache associativa em conjunto com duas linhas por conjunto da Figura 4.15: tamanho do endereço, número de unidades endereçáveis, tamanho de bloco, número de blocos na memória principal, número de linhas no conjunto, número de conjuntos, número de linhas na cache, tamanho da tag.

**4.5** Considere um microprocessador de 32 bits que tem uma cache associativa em conjunto com quatro linhas por conjunto de 16 KBytes no chip. Suponha que a cache tenha um tamanho de linha de quatro palavras de 32 bits. Desenhe um diagrama de blocos dessa cache, mostrando sua organização e como os diferentes campos de endereço são usados para determinar um acerto/falha de cache. Onde, na cache, a palavra no local de memória ABCDE8F8 é mapeada?

**4.6** Dadas as seguintes especificações para uma memória cache externa: associativa em conjunto com quatro vias; tamanho de linha de duas palavras de 16 bits; capaz de acomodar um total de 4 K palavras de 32 bits da memória principal; usada com um processador de 16 bits que emite endereços de 24 bits. Projete a estrutura de cache com todas as informações pertinentes e mostre como ela interpreta os endereços do processador.

**4.7** O Intel 80486 tem uma cache unificada no chip. Ela contém 8 KBytes e tem uma organização associativa em conjunto com quatro linhas por conjunto e um tamanho de bloco de quatro palavras de 32 bits. A cache é organizada em 128 conjuntos. Existe um único “bit válido de linha” e três bits, B0, B1 e B2 (os bits “LRU”), por linha. Em uma falha de cache, o 80486 lê uma linha de 16 bytes da memória principal em apenas uma leitura de memória pelo barramento. Desenhe um diagrama simplificado da cache e mostre como os diferentes campos do endereço são interpretados.

**4.8** Considere uma máquina com uma memória principal endereçável por byte com  $2^{16}$  bytes e um tamanho de bloco de 8 bytes. Suponha que uma cache mapeada diretamente, consistindo em 32 linhas, seja usada com essa máquina.

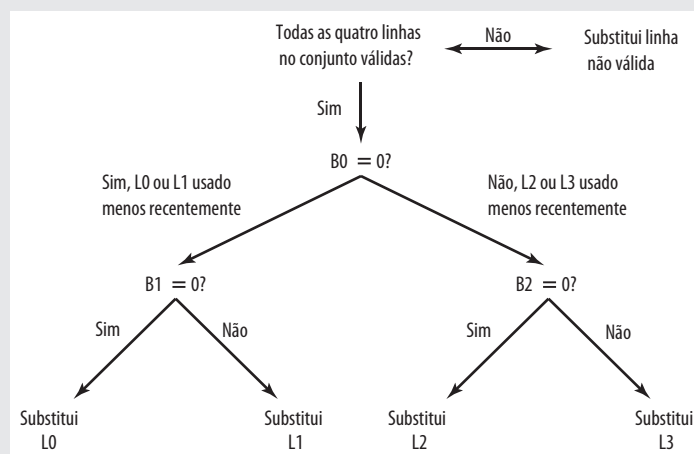
- Como um endereço de memória de 16 bits é dividido em tag, número de linha e número de byte?
- Em que linha seriam armazenados os bytes com cada um dos seguintes endereços?

```
0001 0001 0001 1011
1100 0011 0011 0100
1101 0000 0001 1101
1010 1010 1010 1010
```

- Suponha que o byte com endereço 0001 1010 0001 1010 seja armazenado na cache. Quais são os endereços dos outros bytes armazenados junto com ele?
- Quantos bytes de memória no total podem ser armazenados na cache?
- Por que a tag também é armazenada na cache?

- 4.9** Para sua cache no chip, o Intel 80486 usa um algoritmo de substituição conhecido como **pseudo LRU (pseudo Least Recently Used)**. Associado a cada um dos 128 conjuntos de quatro linhas (rotuladas como L0, L1, L2, L3) existem três bits, B0, B1 e B2. O algoritmo de substituição funciona da seguinte maneira: quando uma linha tiver que ser substituída, a cache primeiro determinará se o uso mais recente foi de L0 e L1 ou L2 e L3. Depois, a cache determinará qual do par de blocos foi usado menos recentemente e o marcará para substituição. A Figura 4.20 ilustra a lógica.
- Especifique como os bits B0, B1 e B2 são definidos e depois descreva, em palavras, como eles são usados no algoritmo de substituição representado na Figura 4.20.
  - Mostre que o algoritmo do 80486 se aproxima de um algoritmo LRU verdadeiro. *Dica:* considere o caso em que a ordem de uso mais recente é L0, L2, L3, L1.
  - Demonstre que um algoritmo LRU verdadeiro exigiria 6 bits por conjunto.
- 4.10** Uma cache associativa em conjunto tem um tamanho de bloco de quatro palavras de 16 bits e um tamanho de conjunto de 2. A cache pode acomodar um total de 4.096 palavras. O tamanho da memória principal que pode ser mantido em cache é de  $64\text{ K} \times 32\text{ bits}$ . Projete a estrutura da cache e mostre como os endereços do processador são interpretados.
- 4.11** Considere um sistema de memória que usa um endereço de 32 bits para endereçar em nível de byte, mais uma cache que usa um tamanho de linha de 64 bytes.
- Considere uma cache mapeada diretamente com um campo de tag no endereço de 20 bits. Mostre o formato de endereço e determine os seguintes parâmetros: número de unidades endereçáveis, número de blocos na memória principal, número de linhas na cache, tamanho da tag.
  - Considere uma cache associativa. Mostre o formato de endereço e determine os seguintes parâmetros: número de unidades endereçáveis, número de blocos na memória principal, número de linhas na cache, tamanho da tag.
  - Considere uma cache associativa em conjunto com quatro linhas por conjunto, com um campo de tag no endereço de 9 bits. Mostre o formato de endereço e determine os seguintes parâmetros: número de unidades endereçáveis, número de blocos na memória principal, número de linhas no conjunto, número de conjuntos na cache, número de linhas na cache, tamanho da tag.
- 4.12** Considere um computador com as seguintes características: total de 1 MByte de memória principal; tamanho de palavra de 1 byte; tamanho de bloco de 16 bytes; e tamanho de cache de 64 KBytes.
- Para os endereços de memória principal F0010, 01234 e CABBE, indique os deslocamentos de tag, endereço de linha de cache e palavra para uma cache mapeada diretamente.
  - Indique dois endereços quaisquer da memória principal com diferentes tags que são mapeados para o mesmo *slot* de cache para uma cache mapeada diretamente.

**Figura 4.20** Estratégia de substituição de cache no chip do Intel 80486



- c. Para os endereços da memória principal F0010 e CABBE, indique os valores correspondentes de tag e deslocamento para uma cache totalmente associativa.
- d. Para os endereços da memória principal F0010 e CABBE, indique os valores correspondentes de tag, conjunto de cache e deslocamento para uma cache associativa em conjunto com duas vias.

**4.13** Descreva uma técnica simples para implementar um algoritmo de substituição LRU em uma cache associativa em conjunto com quatro linhas por conjunto.

**4.14** Considere novamente o Exemplo 4.3. Como a resposta mudaria se a memória principal usasse uma capacidade de transferência em bloco com um tempo de acesso da primeira palavra de 30 ns e um tempo de acesso de 5 ns para cada palavra subsequente?

**4.15** Considere o código a seguir:

```
for (i = 0; i < 20; i++)
    for (j = 0; j < 10; j++)
        a[i] = a[i]*j
```

- a. Dê um exemplo da localidade espacial no código.
- b. Dê um exemplo da localidade temporal no código.

**4.16** Generalize as Equações (4.2) e (4.3), no Apêndice 4A, para hierarquias de memória de  $N$  níveis.

**4.17** Um sistema de computação contém uma memória principal de 32 K palavras de 16 bits. Ele também tem uma cache de 4 K palavras dividida em conjuntos de quatro linhas com 64 palavras por linha. Considere que a cache esteja inicialmente vazia. O processador busca palavras dos locais 0, 1, 2, ..., 4.351, nessa ordem. Depois, ele repete essa sequência de busca mais nove vezes. A cache é 10 vezes mais rápida que a memória principal. Estime a melhoria resultante do uso da cache. Considere uma política de LRU para a substituição em bloco.

**4.18** Considere uma cache de 4 linhas de 16 bytes cada. A memória principal é dividida em blocos de 16 bytes cada. Ou seja, o bloco 0 tem bytes com endereços de 0 a 15, e assim por diante. Agora, considere um programa que acessa a memória na seguinte sequência de endereços:

Uma vez: 63 até 70

Loop dez vezes: 15 até 32; 80 até 95

- a. Suponha que a cache seja organizada como mapeada diretamente. Os blocos de memória 0, 4 e assim por diante são atribuídos à linha 1; os blocos 1, 5 e assim por diante à linha 2; e assim sucessivamente. Calcule a razão de acerto.
- b. Suponha que a cache seja organizada como associativa em conjunto com duas linhas por conjunto, com dois conjuntos de duas linhas cada. Os blocos de numeração par são atribuídos a um conjunto 0 e os blocos de numeração ímpar são atribuídos ao conjunto 1. Calcule a razão de acerto para a cache associativa em conjunto com duas vias usando o esquema de substituição o LRU “usado menos recentemente”.

**4.19** Considere um sistema de memória com os seguintes parâmetros:

$$T_c = 100 \text{ ns} \quad C_c = 10^{-4} \$/\text{bit}$$

$$T_m = 1200 \text{ ns} \quad C_m = 10^{-5} \$/\text{bit}$$

- a. Qual é o custo de 1 MByte de memória principal?
- b. Qual é o custo de 1 MByte de memória principal usando a tecnologia de memória cache?
- c. Se o tempo de acesso efetivo é 10% maior que o tempo de acesso à cache, qual é a razão de acerto  $H$ ?

**4.20** a. Considere uma cache L1 com um tempo de acesso de 1 ns e uma razão de acerto de  $H = 0,95$ . Suponha que possamos mudar o projeto da cache (tamanho da cache, organização) de modo que aumentemos  $H$  para 0,97, mas aumentando o tempo de acesso para 1,5 ns. Quais condições deverão ser atendidas para que essa mudança resulte em um desempenho melhorado?

- b. Explique por que esse resultado faz sentido intuitivamente.

**4.21** Considere uma cache de um único nível com um tempo de acesso de 2,5 ns, um tamanho de linha de 64 bytes e uma razão de acerto de  $H = 0,95$ . A memória principal usa uma capacidade de transferência em bloco que tem um tempo de acesso da primeira palavra (4 bytes) de 50 ns e um tempo de acesso de 5 ns para cada palavra subsequente.

- a. Qual é o tempo de acesso quando existe uma falha de cache? Suponha que a cache espere até que a linha tenha sido buscada da memória principal e depois reexecute para um acerto.
- b. Suponha que aumentar o tamanho da linha para 128 bytes aumente o  $H$  para 0,97. Isso reduz o tempo de acesso médio à memória?

**4.22** Um computador tem uma cache, uma memória principal e um disco usado para memória virtual. Se uma palavra referenciada estiver na cache, 20 ns são necessários para acessá-la. Se estiver na memória principal, mas não na cache, 60 ns são necessários para carregá-la para a cache, e depois a referência é iniciada novamente. Se a palavra não estiver na memória principal, 12 ms são necessários para buscar a palavra do disco,



seguidos por 60 ns para copiá-la para a cache e depois a referência é iniciada novamente. A razão de acerto da cache é 0,9 e a razão de acerto da memória principal é 0,6. Qual é o tempo médio em nanossegundos necessário para acessar uma palavra referenciada nesse sistema?

- 4.23** Considere uma cache com um tamanho de linha de 64 bytes. Suponha que, na média, 30% das linhas na cache estejam modificadas. Uma palavra consiste em 8 bytes.
- Suponha que haja uma taxa de falha de 3% (razão de acerto de 0,97). Calcule a quantidade de tráfego da memória principal, em termos de bytes por instrução para políticas *write-through* e *write-back*. A memória é lida na cache uma linha por vez. Porém, para *write-back*, uma única palavra pode ser escrita da cache para a memória principal.
  - Repita a parte (a) para uma taxa de 5%.
  - Repita a parte (a) para uma taxa de 7%.
  - Com esses resultados, a que conclusão você pode chegar?
- 4.24** No microprocessador Motorola 68020, um acesso à cache leva dois ciclos de clock. O acesso a dados da memória principal pelo barramento até o processador leva três ciclos de clock no caso de nenhuma inserção de estado de espera; os dados são entregues ao processador em paralelo com entrega à cache.
- Calcule o tamanho efetivo de um ciclo de memória dada uma razão de acerto de 0,9 e uma frequência de clock de 16,67 MHz.
  - Repita os cálculos considerando a inserção de dois estados de espera de um ciclo cada por ciclo de memória. Com esses resultados, a que conclusão você pode chegar?
- 4.25** Considere um processador que possui um tempo de ciclo de memória de 300 ns e uma taxa de processamento de instrução de 1 MIPS. Na média, cada instrução requer um ciclo de memória do barramento para busca de instrução e um para o operando envolvido.
- Calcule a utilização do barramento pelo processador.
  - Suponha que o processador esteja equipado com uma cache de instruções e a razão de acerto associada seja 0,5. Determine o impacto sobre a utilização do barramento.
- 4.26** O desempenho de um sistema de cache de um único nível para uma operação de leitura pode ser caracterizado pela seguinte equação:

$$T_a = T_c + (1 - H)T_m$$

onde  $T_a$  é o tempo médio de acesso,  $T_c$  é o tempo de acesso à cache,  $T_m$  é o tempo de acesso à memória (memória ao registrador do processador) e  $H$  é a razão de acerto. Para simplificar, consideramos que a palavra em questão é carregada na cache em paralelo com o load para o registrador do processador. Essa é a mesma forma da Equação 4.2.

- Defina  $T_b$  = tempo para transferir uma linha entre a cache e a memória principal, e  $W$  = fração de referências de escrita. Revise a equação anterior para considerar as escritas e também as leituras, usando uma política *write-through*.
  - Defina  $W_b$  como a probabilidade de que uma linha na cache tenha sido alterada. Ofereça uma equação para  $T_a$  para a política *write-back*.
- 4.27** Para um sistema com dois níveis de cache, defina  $T_{c1}$  = tempo de acesso da cache de primeiro nível;  $T_{c2}$  = tempo de acesso da cache de segundo nível;  $T_m$  = tempo de acesso à memória;  $H_1$  = razão de acerto da cache de primeiro nível;  $H_2$  = razão de acerto da cache de primeiro e segundo níveis combinadas. Ofereça uma equação para  $T_a$  para uma operação de leitura.
- 4.28** Considere as seguintes características de desempenho em uma falha de leitura de cache: um ciclo de clock para enviar um endereço à memória principal e quatro ciclos de clock para acessar uma palavra de 32 bits da memória principal e transferi-la para o processador e a cache.
- Se o tamanho da linha de cache for uma palavra, qual é a penalidade de falha (ou seja, o tempo adicional exigido para uma leitura no evento de uma falha de leitura)?
  - Qual é a penalidade de falha se um tamanho de linha de cache tiver quatro palavras e for executada uma transferência múltipla, não em rajada?
  - Qual é a penalidade de falha se o tamanho da linha de cache for quatro palavras e uma transferência for executada, com um ciclo de clock por transferência de palavra?
- 4.29** Para o projeto de cache do problema anterior, suponha que aumentar o tamanho da linha de uma palavra para quatro palavras resulta em uma diminuição da taxa de falha de leitura de 3,2% para 1,1%. Para os casos de transferência sem rajada e com rajada, qual é a penalidade de falha média, considerando todas as leituras, para os dois tamanhos de linha diferentes?



## Apêndice 4A Características de desempenho das memórias de dois níveis

Neste capítulo, é feita uma referência a uma cache que atua como um buffer entre a memória principal e o processador, criando uma memória interna de dois níveis. Essa arquitetura de dois níveis explora uma propriedade conhecida como localidade, para oferecer melhor desempenho em relação a uma memória de um nível comparável.

O mecanismo de cache da memória principal faz parte da arquitetura do computador, implementada no hardware e normalmente invisível ao sistema operacional. Existem dois outros casos de uma técnica de memória de dois níveis que também exploram a localidade e que são, pelo menos parcialmente, implementadas no sistema operacional: memória virtual e a cache de disco (Tabela 4.7). A memória virtual é explorada no Capítulo 8; a cache de disco está fora do escopo deste livro, mas é examinada em Stallings (2009<sup>b</sup>). Neste apêndice, examinamos algumas das características de desempenho das memórias de dois níveis que são comuns às três técnicas.



## Localidade

A base para a vantagem de desempenho de uma memória de dois níveis é um princípio conhecido como **localidade de referência** (DENNING, 1968<sup>a</sup>). Esse princípio afirma que as referências à memória tendem a se agrupar. Por um longo período, os agrupamentos em uso mudam, mas por um período curto, o processador está trabalhando principalmente com grupos fixos de referências à memória.

Intuitivamente, o princípio de localidade faz sentido. Considere a seguinte linha de raciocínio:

1. Com exceção das instruções de desvio e chamada, que constituem apenas uma pequena fração de todas as instruções do programa, a execução do programa é sequencial. Logo, na maior parte dos casos, a próxima instrução a ser apanhada vem imediatamente após a última instrução apanhada.
2. É raro ter uma longa sequência ininterrupta de chamadas de procedimento seguidas pela sequência de retornos correspondente. Em vez disso, um programa permanece confinado a uma janela um tanto estreita de profundidade de chamada de procedimento. Assim, por um curto período, as referências às instruções tendem a estar localizadas em alguns poucos procedimentos.
3. A maioria das construções iterativas consiste em um número relativamente pequeno de instruções repetidas muitas vezes. Pela duração da iteração, o cálculo é, portanto, confinado a uma pequena parte contígua de um programa.
4. Em muitos programas, grande parte do cálculo envolve processamento de estruturas de dados, como arrays ou sequências de registros. Em muitos casos, referências sucessivas a essas estruturas de dados serão localizadas a itens de dados próximos.

Essa linha de raciocínio tem sido confirmada em muitos estudos. Com referência ao ponto 1, diversos estudos têm analisado o comportamento dos programas em linguagem de alto nível. A Tabela 4.8 inclui os principais resultados, medindo o surgimento de vários tipos de instrução durante a execução, a partir dos estudos a seguir. O estudo mais antigo do comportamento da linguagem de programação, realizado por Knuth (1971<sup>bb</sup>), examinou uma coleção de programas FORTRAN usados como exercícios para alunos. Tanenbaum (1978<sup>c</sup>) publicou medições coletadas de mais de 300 procedimentos usados em programas de sistema operacional e escritos em uma linguagem que aceita programação estruturada (SAL). Patterson e Sequin (1982<sup>dd</sup>) analisaram um conjunto de medições tomadas de compiladores e programas para editoração, projeto auxiliado por computador (CAD), classificação e comparação de arquivos. As linguagens de programação C e Pascal foram estudadas. Huck (1983<sup>ee</sup>) analisou quatro programas que resembram uma mistura de computação científica de uso geral, incluindo transformação rápida de Fourier e a integração de sistemas de equações diferenciais. Existe um acordo tão bom sobre os resultados dessa mistura de linguagens e aplicações que as instruções de desvio e chamada representam apenas uma fração das instruções executadas durante o tempo de vida de um programa. Assim, esses estudos confirmam a afirmação 1.

**Tabela 4.7** Características das memórias de dois níveis

|                                        | Cache da memória principal         | Memória virtual (paginação)                     | Cache de disco                                  |
|----------------------------------------|------------------------------------|-------------------------------------------------|-------------------------------------------------|
| Razões típicas de tempo de acesso      | 5 : 1 (memória principal × cache)  | 10 <sup>6</sup> : 1 (memória principal × disco) | 10 <sup>6</sup> : 1 (memória principal × disco) |
| Sistema de gerenciamento de memória    | Implementada por hardware especial | Combinação de hardware e software do sistema    | Software do sistema                             |
| Tamanho típico de bloco ou página      | 4 a 128 bytes (bloco de cache)     | 64 a 4096 bytes (página de memória virtual)     | 64 a 4096 bytes (bloco ou páginas do disco)     |
| Acesso do processador ao segundo nível | Acesso direto                      | Acesso indireto                                 | Acesso indireto                                 |

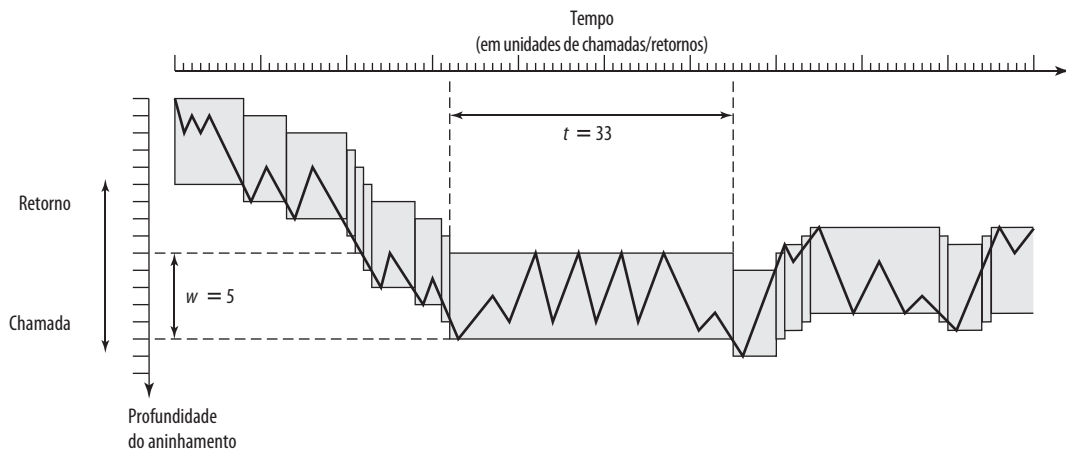
**Tabela 4.8** Frequência dinâmica relativa das operações em linguagens de alto nível

| Carga de trabalho da linguagem em estudo | HUCK, 1983 <sup>ee</sup><br>Pascal Científico | KNUTH, 1971 <sup>bb</sup><br>FORTRAN Aluno | Patterson e Sequin, 1982 <sup>dd</sup> |           | Tanenbaum, 1978 <sup>cc</sup><br>Sistema SAL |
|------------------------------------------|-----------------------------------------------|--------------------------------------------|----------------------------------------|-----------|----------------------------------------------|
|                                          |                                               |                                            | Sistema Pascal                         | Sistema C |                                              |
| Assinalamento                            | 74                                            | 67                                         | 45                                     | 38        | 42                                           |
| Loop                                     | 4                                             | 3                                          | 5                                      | 3         | 4                                            |
| Call                                     | 1                                             | 3                                          | 15                                     | 12        | 12                                           |
| IF                                       | 20                                            | 11                                         | 29                                     | 43        | 36                                           |
| GOTO                                     | 2                                             | 9                                          | —                                      | 3         | —                                            |
| Outras                                   | —                                             | 7                                          | 6                                      | 1         | 6                                            |

Com relação à afirmação 2, os estudos relatados em Patterson (1985<sup>ii</sup>) confirmam. Isso é ilustrado na Figura 4.21, que mostra o comportamento de chamada-retorno. Cada chamada é representada pela linha descendo para a direita, e cada retorno pela linha subindo para a direita. Na figura, uma janela com profundidade igual a 5 é definida. Somente uma sequência de chamadas e retornos com um movimento de profundidade 6 em qualquer direção faz com que a janela se mova. Como podemos ver, o programa em execução pode permanecer dentro de uma janela estacionária por longos períodos. Um estudo pelos mesmos analistas de programas C e Pascal mostrou que uma janela de profundidade 8 precisará se deslocar apenas em menos de 1% das chamadas ou retornos (TAMIR e SEQUIN, 1983<sup>gg</sup>).

A literatura faz uma distinção entre localidade espacial e localidade temporal. A localidade espacial refere-se à tendência da execução de envolver uma série de locais de memória que estão agrupados. Isso reflete a tendência de um processador de acessar as instruções sequencialmente. A localidade espacial também reflete a tendência de um programa de acessar locais de dados sequencialmente, como ao processar uma tabela de dados. A localidade temporal refere-se à tendência de um processador de acessar locais de memória que foram usados recentemente. Por exemplo, quando um loop é executado, o processador executa o mesmo conjunto de instruções repetidamente.

Tradicionalmente, a localidade temporal é explorada mantendo valores de dados e instruções usados recentemente na memória cache e explorando uma hierarquia de cache. A localidade espacial geralmente é explorada usando blocos de cache maiores e incorporando mecanismos de pré-busca (buscando itens de uso antecipado) na lógica de controle de cache. Recentemente, tem havido uma pesquisa considerável sobre o refinamento dessas técnicas para alcançar maior desempenho, mas as estratégias básicas continuam sendo as mesmas.

**Figura 4.21** Exemplo de comportamento de chamada-retorno de um programa



## Operação da memória de dois níveis

A propriedade de localidade pode ser explorada na formação de uma memória de dois níveis. A memória de nível superior (M1) é menor, mais rápida e mais cara (por bit) do que a memória de nível inferior (M2). M1 é usada como um armazenamento temporário para parte do conteúdo da M2 maior. Quando é feita uma referência à memória, é feita uma tentativa de acessar o item em M1. Se isso tiver sucesso, então é feito um acesso rápido. Se não, então um bloco de locais de memória é copiado de M2 para M1, e o acesso então ocorre através de M1. Devido à localidade, quando um bloco é trazido para M1, deverá haver uma série de acessos a locais nesse bloco, resultando em um atendimento geral mais rápido.

Para expressar o tempo médio para acessar um item, temos que considerar não apenas as velocidades dos dois níveis de memória, mas também a probabilidade de que determinada referência possa ser encontrada em M1. Temos:

$$\begin{aligned} T_s &= H \times T_1 + (1 - H) \times (T_1 + T_2) \\ &= T_1 + (1 - H) \times T_2 \end{aligned} \quad (4.2)$$

onde

$T_s$  = tempo de acesso médio (sistema)

$T_1$  = tempo de acesso de M1 (por exemplo, cache, cache de disco)

$T_2$  = tempo de acesso de M2 (por exemplo, memória principal, disco)

$H$  = razão de acerto (fração do tempo em que é encontrada uma referência em M1)

A Figura 4.2 mostra o tempo de acesso médio como uma função da razão de acerto. Como podemos ver, para uma alta percentagem de acertos, o tempo médio de acesso total é muito mais próximo daquele de M1 do que de M2.



## Desempenho

Vejam alguns dos parâmetros relevantes a uma avaliação de um mecanismo de memória de dois níveis. Primeiro, considere o custo. Temos:

$$C_s = \frac{C_1 S_1 + C_2 S_2}{S_1 + S_2} \quad (4.3)$$

onde

$C_s$  = custo médio por bit para a memória de dois níveis combinada

$C_1$  = custo médio por bit da memória de nível superior M1

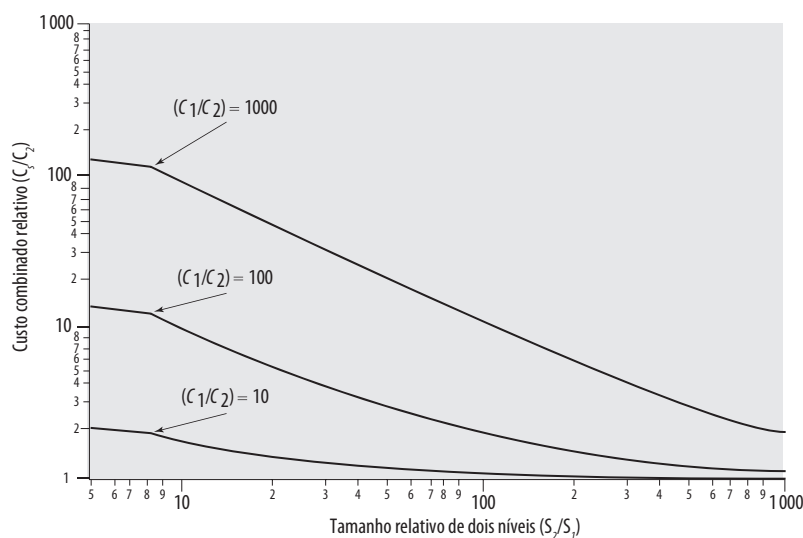
$C_2$  = custo médio por bit da memória de nível inferior M2

$S_1$  = tamanho de M1

$S_2$  = tamanho de M2

Gostariamos que  $C_s \approx C_2$ . Dado que  $C_1 \gg C_2$ , isso requer  $S_1 \ll S_2$ . A Figura 4.22 mostra o relacionamento.

**Figura 4.22** Relacionamento do custo de memória médio com tamanho de memória relativo para uma memória de dois níveis



Em seguida, considere o tempo de acesso. Para uma memória de dois níveis oferecer uma melhoria de desempenho significativa, precisamos ter  $T_s$  aproximadamente igual a  $T_1$  ( $T_s \approx T_1$ ). Dado que  $T_1$  é muito menor que  $T_2$  ( $T_1 \ll T_2$ ), uma razão de acerto próxima de 1 é necessária.

Assim, gostaríamos que M1 fosse pequena para reduzir o custo, e grande para melhorar a razão de acerto e, portanto, o desempenho. Existe um tamanho de M1 que satisfaça aos dois requisitos a um grau razoável? Podemos responder a essa pergunta com uma série de subperguntas:

- Que valor de razão de acerto é necessário para que  $T_s \approx T_1$ ?
- Que tamanho de M1 garantirá a razão de acerto necessária?
- Esse tamanho satisfaz o requisito de custo?

Para conseguir isso, considere a quantidade  $T_1/T_s$ , que é conhecida como *eficiência de acesso*. Essa é uma medida da proximidade entre o tempo de acesso médio ( $T_s$ ) e o tempo de acesso de M1 ( $T_1$ ). Pela Equação (4.2),

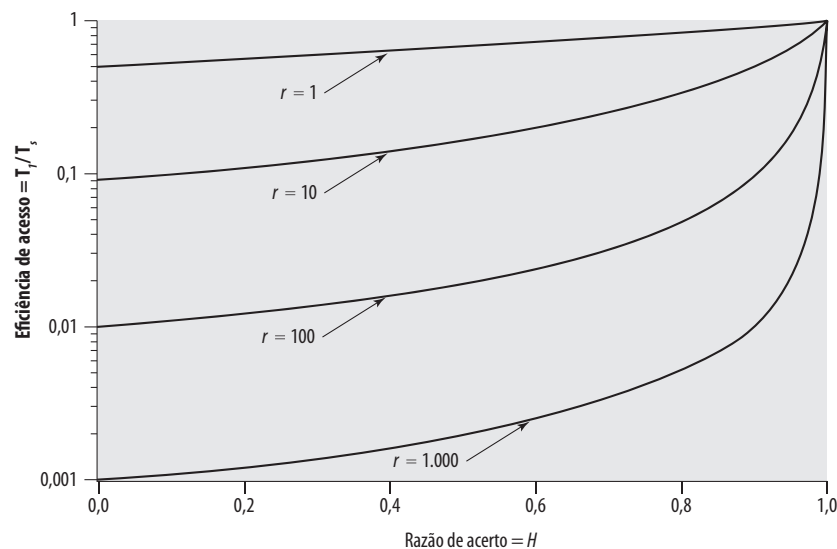
$$\frac{T_1}{T_s} = \frac{1}{1 + (1 - H) \frac{T_2}{T_1}} \quad (4.4)$$

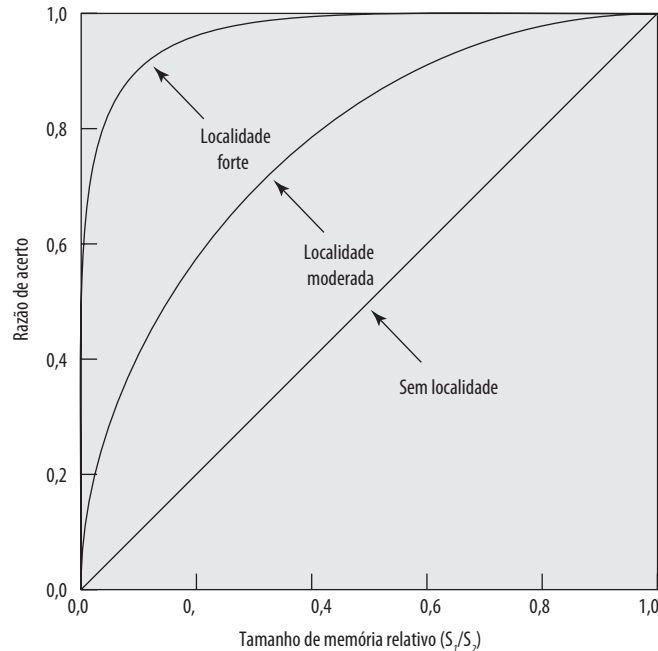
A Figura 4.23 representa graficamente  $T_1/T_s$  como uma função da razão de acerto  $H$ , com a quantidade  $T_2/T_1$  como um parâmetro. Normalmente, o tempo de acesso à cache no chip é de cerca de 25 a 50 vezes mais rápido que o tempo de acesso à memória principal (ou seja,  $T_2/T_1$  é 5 a 10), o tempo de acesso à cache fora do chip é cerca de 5 a 15 vezes mais rápido que o tempo de acesso à memória principal (ou seja,  $T_2/T_1$  é 5 a 15), e o tempo de acesso à memória principal é cerca de 1.000 vezes mais rápido que o tempo de acesso ao disco ( $T_2/T_1 = 1.000$ ). Assim, uma razão de acerto na faixa de algo em torno de 0,9 parece ser necessária para satisfazer o requisito de desempenho.

Agora, podemos formular a questão sobre o tamanho relativo da memória com mais exatidão. Uma razão de acerto de, digamos, 0,8 ou melhor, é razoável para  $S_1 \ll S_2$ ? Isso dependerá de diversos fatores, incluindo a natureza do software sendo executado e dos detalhes do projeto da memória de dois níveis. O principal determinante, logicamente, é o grau de localidade. A Figura 4.24 sugere o efeito que a localidade tem sobre a razão de acerto. Claramente, se M1 tiver o mesmo tamanho que M2, então a razão de acerto será 1,0: todos os itens em M2 sempre são armazenados também em M1. Agora, suponha que não haja localidade, ou seja, as referências são completamente aleatórias. Nesse caso, a razão de acerto deverá ser uma função estritamente linear do tamanho relativo da memória. Por exemplo, se M1 tiver a metade do tamanho de M2, então, a qualquer momento, metade dos itens de M2 também estão em M1, e a razão de acerto será 0,5. Na prática, porém, existe algum grau de localidade nas referências. Os efeitos da localidade moderada e forte são indicados na figura. Observe que a Figura 4.24 não é derivada de qualquer dado ou modelo específico; a figura sugere o tipo de desempenho que é visto com diversos graus de localidade.

Assim, se houver localidade forte, é possível alcançar altos valores de razão de acerto, mesmo com um tamanho de memória de nível superior relativamente pequeno. Por exemplo, diversos estudos têm mostrado que tamanhos de cache pequenos gerarão uma razão de acerto acima de 0,75, independentemente do tamanho da memória principal [por exemplo, Agarwal (1989<sup>a</sup>); Przybylski, Horowitz e Hennessy (1988<sup>o</sup>)];

**Figura 4.23** Eficiência de acesso como uma função da taxa de acerto ( $r = T_2/T_1$ )



**Figura 4.24** Razão de acerto como uma função do tamanho de memória relativo

Strecker (1983<sup>hh</sup>) e Smith(1992<sup>m</sup>)]<sup>xlv</sup> Uma cache na faixa de 1 K a 128 K palavras geralmente é adequada, enquanto a memória principal agora normalmente está na faixa dos gigabytes. Quando considerarmos a memória virtual e a cache de disco, citaremos outros estudos que confirmam o mesmo fenômeno, especialmente que um M1 relativamente pequeno gera um valor alto de razão de acerto, devido à localidade.

Isso nos leva à última pergunta listada anteriormente: o tamanho relativo das duas memórias satisfaz o requisito de custo? A resposta é clara: sim. Se só precisarmos de uma memória de nível superior relativamente pequena para conseguir um bom desempenho, então o custo médio por bit dos dois níveis de memória se aproximará do menor custo da memória de nível inferior.

Por favor, observe que, com o envolvimento de cache L2, ou ainda de caches L2 e L3, a análise é muito mais complexa. Veja algumas discussões em Peir, Hsu e Smith (1999<sup>i</sup>) e Handy (1993<sup>g</sup>).

## Referências

- a DENNING, P. "The working set model for program behavior". *Communications of the ACM*, mai. 1968.
- b STALLINGS, W. *Operating systems, internals and design principles*, 6<sup>th</sup> Edition. Upper Saddle River, NJ: Prentice Hall, 2009.
- c BAILEY, D. "RISC microprocessors and scientific computing". *Proceedings, Supercomputing '93*, 1993.
- d WANG, G. e TAFTI, D. "Performance enhancement on microprocessors with hierarchical memory systems for solving large sparse linear systems". *International Journal of Supercomputing Applications*, vol. 13, 1999.
- e PRESSEL, D. "Fundamental limitations on the use of prefetching and stream buffers for scientific applications". *Proceedings, ACM Symposium on Applied Computing*, mar. 2001.
- f DOWD, K. e SEVERANCE, C. *High performance computing*. Sebastopol, CA: O'Reilly, 1998.
- g CEKLEOV, M. e DUBOIS, M. "Virtual-address caches, part 1: problems and solutions in uniprocessors". *IEEE Micro*, set./out. 1997.
- h JACOB, B.; NG, S. e WANG, D. *Memory systems: cache, DRAM, disk*. Boston: Morgan Kaufmann, 2008.
- i MAYBERRY, W. e EFLAND, G. "Cache boosts multiprocessor performance". *Computer Design*, nov. 1984.

- j HILL, M. "Evaluating associativity in CPU caches". *IEEE Transactions on Computers*, dez. 1989.
- k GENU, P. *A cache primer*. Application note AN2663. Freescale Semiconductor, Inc., 2004. Disponível em: <[www.freescale.com/files/32bit/doc/app\\_note/AN2663.pdf](http://www.freescale.com/files/32bit/doc/app_note/AN2663.pdf)>.
- l CANTIN, J. e HILL, H. "Cache performance for selected SPEC CPU2000 benchmarks". *Computer Architecture News*, set. 2001.
- m SMITH, A. "Cache memories". *ACM Computing Surveys*, set. 1982.
- n SMITH, A. "Line (block) size choice for CPU cache memories". *IEEE Transactions on Communications*, set. 1987.
- o PRZYBYLSKI, S.; HOROWITZ, M. e HENNESSY, J. "Performance trade-offs in cache design". *Proceedings, 15th Annual International Symposium on Computer Architecture*, jun. 1988.
- p PRZYBYLSKI, S. "The performance impact of block size and fetch strategies". *Proceedings, 17th Annual International Symposium on Computer Architecture*, maio de 1990.
- q HANDY, J. *The cache memory book*. San Diego: Academic Press, 1993.
- r AZIMI, M.; PRASAD, B. e BHAT, K. "Two level cache architectures". *Proceedings COMPCON '92*, fev. 1992.
- s NOVITSKY, J.; AZIMI, M. e GHAZNAVI, R. "Optimizing systems performance based on pentium processors". *Proceedings COMPCON '92*, fev. 1993.
- t PEIR, J.; HSU, W.; e SMITH, A. "Functional implementation techniques for CPU cache memories". *IEEE Transactions on Computers*, fev. 1999.
- u GHAI, S.; JOYNER, J. e JOHN, L. *Investigating the effectiveness of a third level cache*. Technical Report TR-980501-01, Laboratory for Computer Architecture, University of Texas at Austin. Disponível em: <<http://lca.ece.utexas.edu/pubs-by-type.html>>.
- v SLOSS, A.; SYMES, D. e WRIGHT, C. *ARM System Developer's Guide*. San Francisco: Morgan Kaufmann, 2004.
- w WILKES, M. "Slave memories and dynamic storage allocation", *IEEE Transactions on Electronic Computers*, abril de 1965. Reimpresso em Hill, Touppi e Sohi, 2000.
- x GOODMAN, J. "Using cache memory to reduce processor-memory bandwidth". *Proceedings, 10th Annual International Symposium on Computer Architecture*, 1983. Reimpresso em Hill, Touppi e Sohi, 2000.
- y BELL, J.; CASASENT, D. e BELL, C. "An investigation into alternative cache organizations". *IEEE Transactions on Computers*, abril de 1974. Disponível em: <<http://research.microsoft.com/users/GBell/gbvita.htm>>.
- z AGARWAL, A. *Analysis of cache performance for operating systems and multiprogramming*. Boston: Kluwer Academic Publishers, 1989.
- aa HIGBIE, L. "Quick and easy cache performance analysis". *Computer Architecture News*, jun. 1990.
- bb KNUTH, D. "An empirical study of FORTRAN programs". *Software Practice and Experience*, vol. 1, 1971.
- cc TANENBAUM, A. "Implications of structured programming for machine architecture". *Communications of the ACM*, março de 1978.
- dd PATTERSON, D. e SEQUIN, C. "A VLSI RISC". *Computer*, set. 1982.
- ee HUCK, T. *Comparative analysis of computer architectures*. Stanford University Technical Report No. 83-243, maio de 1983.
- ff PATTERSON, D. "Reduced instruction set computers". *Communications of the ACM*. jan. 1985.
- gg TAMIR, Y. e SEQUIN, C. "Strategies for managing the register file in RISC". *IEEE Transactions on Computers*, novembro de 1983.
- hh STRECKER, W. "Transient behavior of cache memories". *ACM Transactions on Computer Systems*, nov. 1983.
- ii HILL, M.; Touppi, N. e SOHI G. *Readings in computer architecture*. San Francisco: Morgan Kaufmann, 2000.